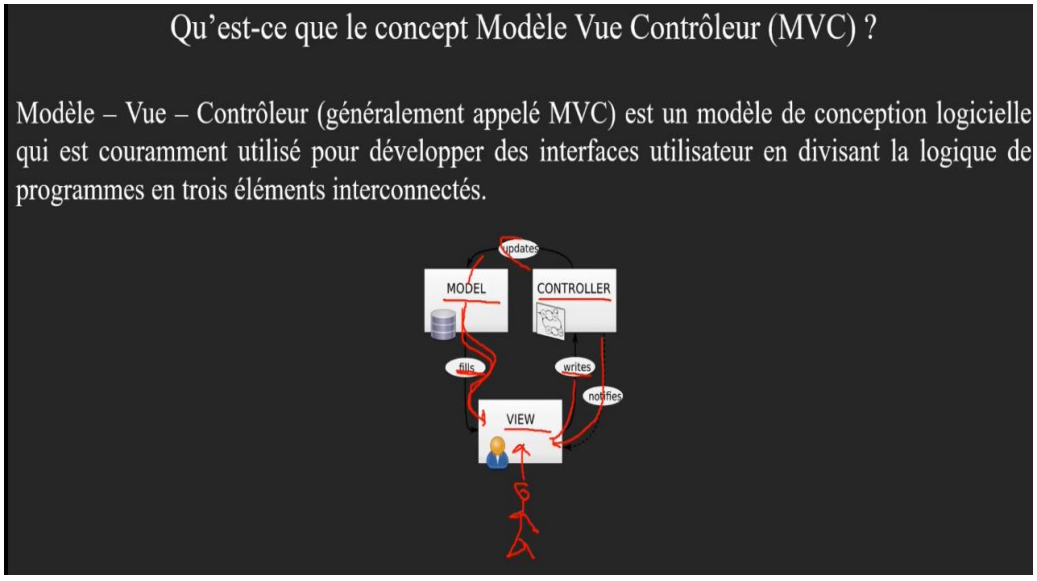
Section 1 : Introduction

1. Installations d'environnements



2. Modèle Vue Contrôleur





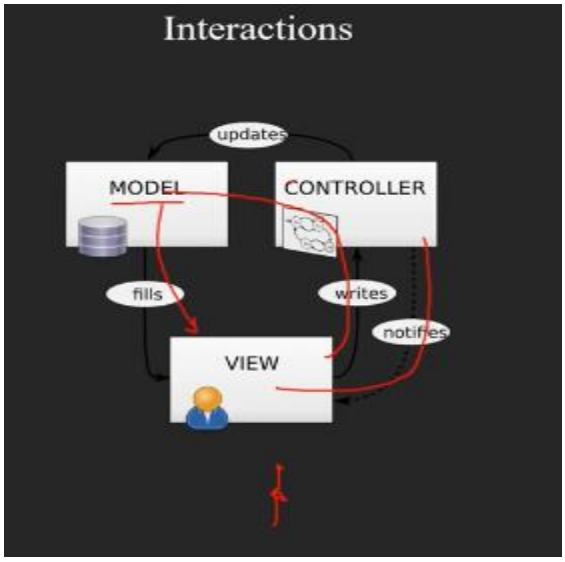
- Le modèle : représente la base de données
- Les Views : la vue, tout le contenu de votre navigateur (les formulaires, les différentes pages.
  - Le Contrôleur : interface entre le modèle et la vue, c'est le chef d'orchestre comme on le dit souvent.

Les Composants du MVC

Le modèle : le composant central du modèle. Il s'agit de la structure de données dynamique de l'application, indépendante de l'interface utilisateur. Il gère directement les données, logique et règles de l'application (un modèle contient les données à visualiser).

La vue : Partie visible d'une interface graphique. La vue se sert du modèle, et peut être diagramme, un formulaire, des boutons, etc. Une vue contient des éléments visuels ainsi que la logique nécessaire pour afficher les données provenant du modèle. Dans une application de bureau classique, la vue obtient les données nécessaires à la présentation du modèle en posant des questions. Elle peut également mettre à jour le modèle en envoyant des messages appropriés. Dans une application web une vue contient des balises HTML.

Le contrôleur : Module qui traite les actions de l'utilisateur, modifie les données du modèle et de la vue.



3. Route (PARTIE 1)

https://laravel.com/docs/10.x

Création du projet :

Allez sur le disque **c:** puis dans le dossier **xammp** puis dans le dossier**htdocs** et c'est à ce niveau que nous allons créer notre projet :**basiclessons**

Il existe deux manières de créer un projet Laravel :

Première méthode :

```
composer create-project laravel/laravel:^10.0 example-app
```

10.0 : la version utilisée

example-app : le nom du projet créé

Deuxième méthode :

```
composer global require laravel/installer

laravel new example-app
```

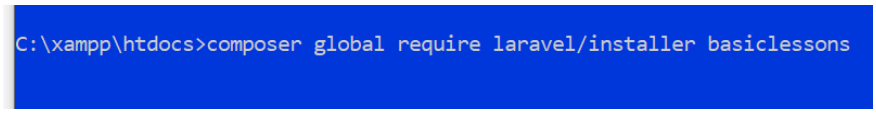
Ensuite : se déplacer dans le dossier nouvellement créé et lancer le serveur.

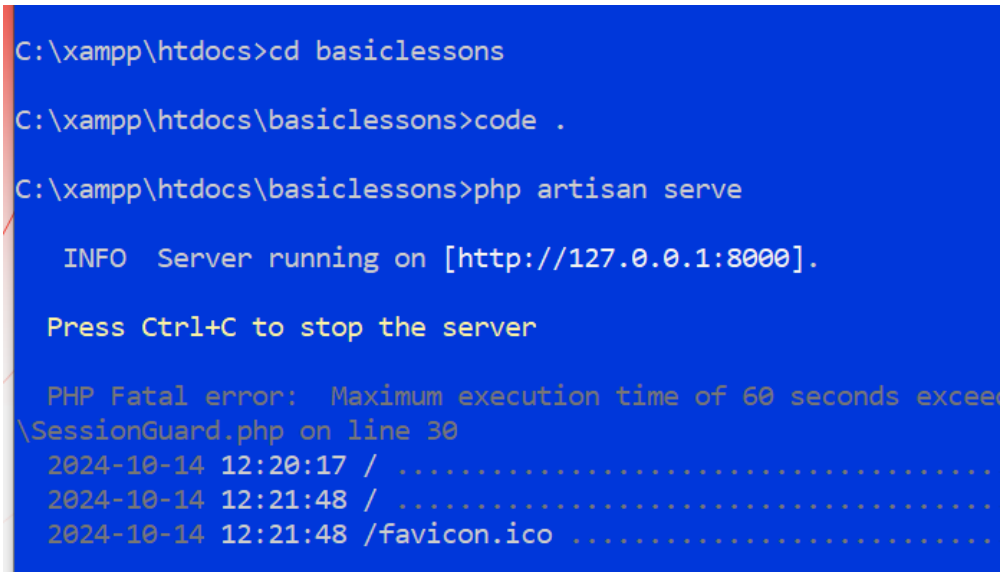
```
cd example-app

php artisan serve
```

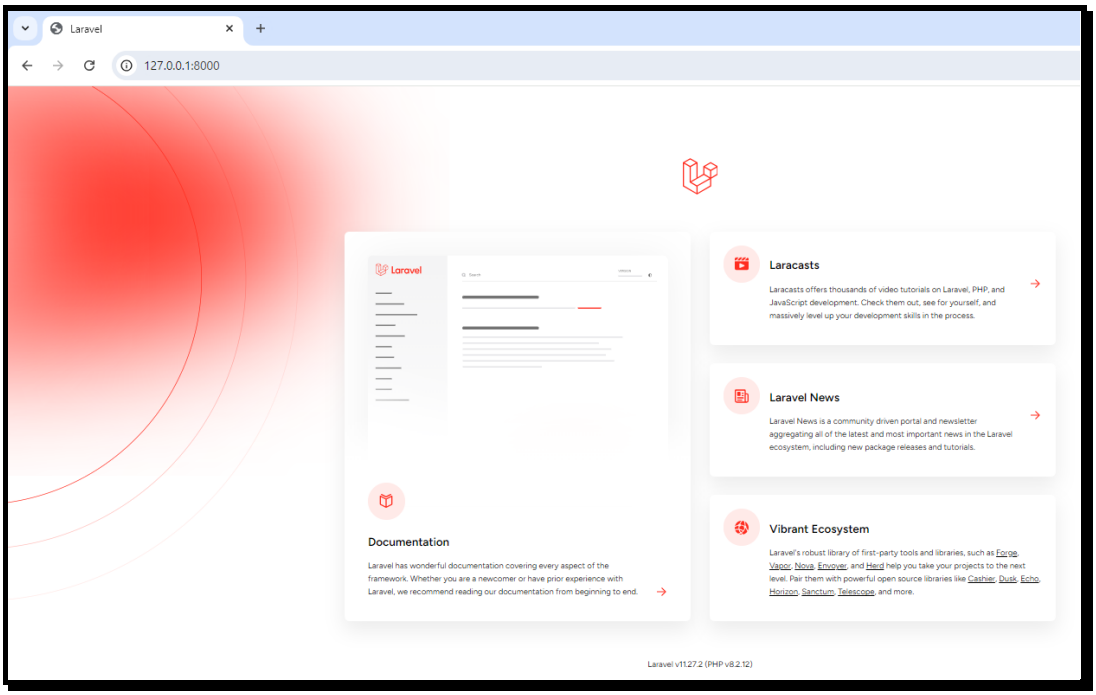
Nous allons utiliser la deuxième méthode pour créer notre projet : ***basiclessons*** :





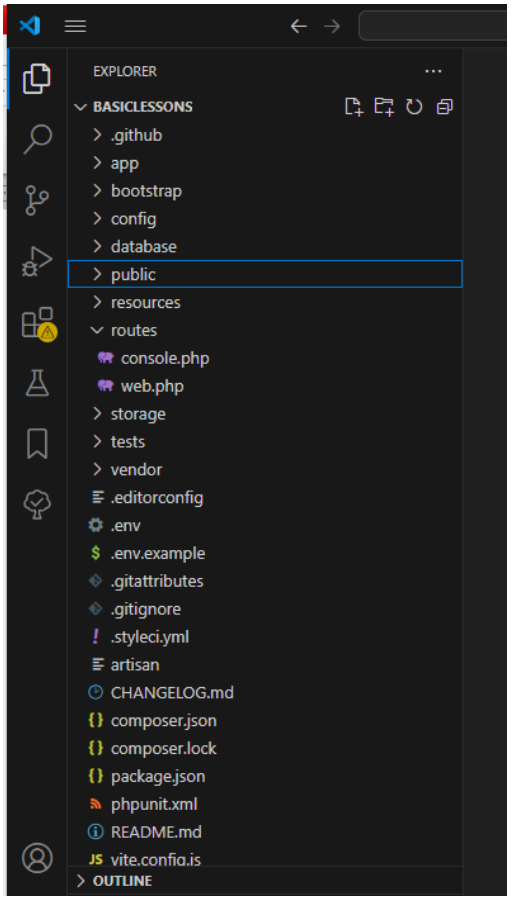


Dans le navigateur :

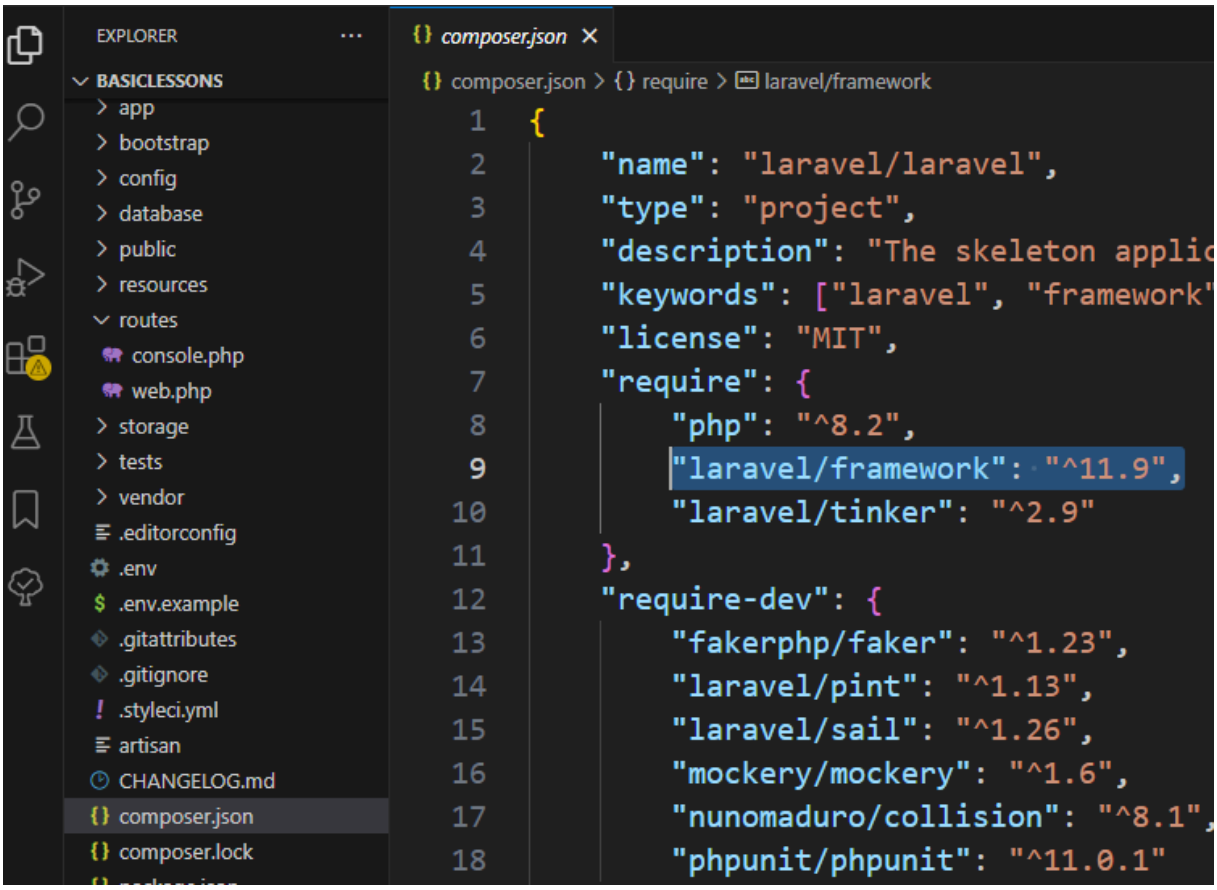


La page est celle par défaut :
Nous avons la version 11.27.2 de Laravel et (PHP v8.2.12).

-Dans VS Code :



La version installée est affichée dans le fichier composer.json :



Explication du système de ***routing*** (routage) :

Notre page par défaut est obtenue à l'aide de la route ci-dessous :



Explication :
La méthode **get()** : prend en paramètre l'url et en second paramètre une fonction qui retourne une **vue** qui est en réalité un fichier contenu dans le dossier **/ressources/views.** Ce fichier est *welcome.blade.php*.
NB : tous les fichiers dans Laravel ont pour extension ***.blade.php***

Dans notre cas c'est '/' qui permet d'afficher notre vue (welcome.blade.php).

Exemple :

Créons une route de la sorte :

```php
<?php

use Illuminate\Support\Facades\Route;

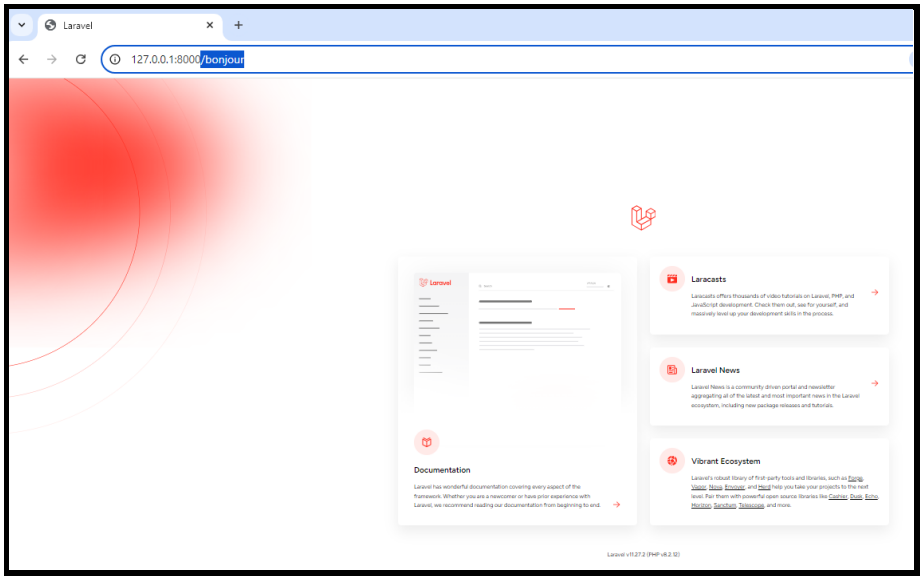
// Route::get('/', function () {
//     return view('welcome');
// });

Route::get('/bonjour', function () {
    return view('welcome');
});
```

- Actualisons le navigateur :

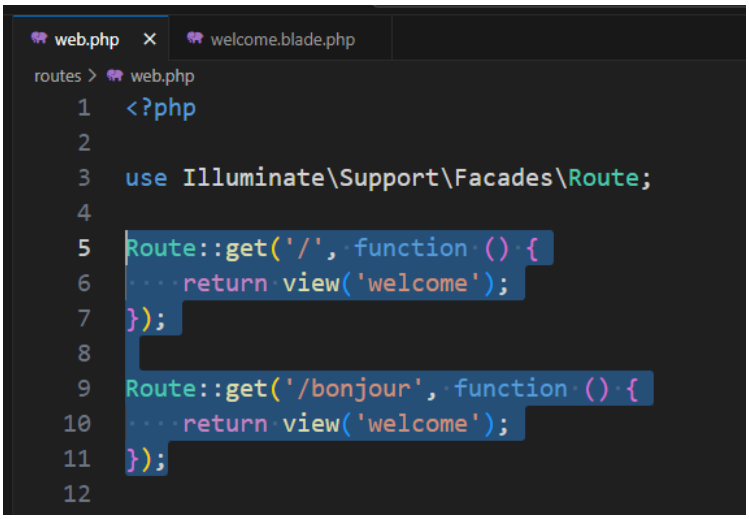


- **/bonjour** dans l'url :

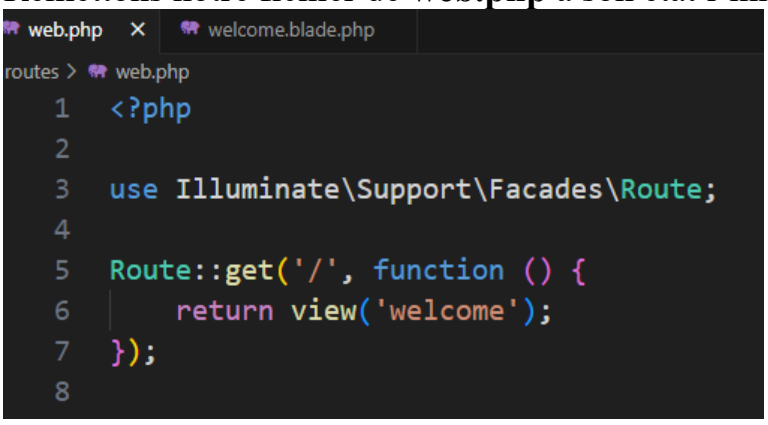


La page par défaut s'affiche.

Maintenant :



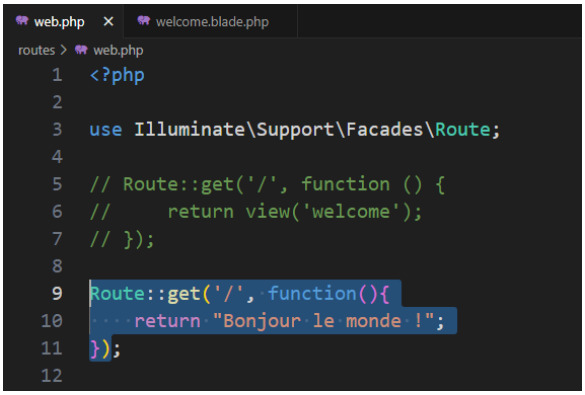
On remarque à une même page correspondent deux routes différentes.
Remettons notre fichier de **web.php** à son état l'initial.



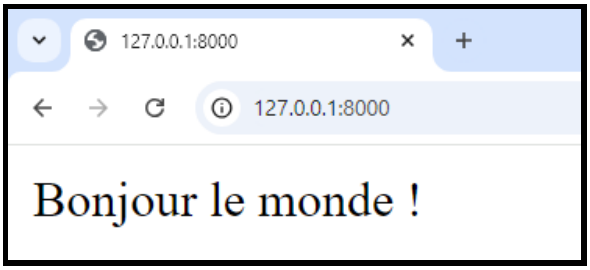
4. Route (PARTIE 2)

Autant d'urls correspondent à autant de vues.
Nous allons créer nos propres routes :
D'autres méthodes autre que ***get*** existent : ***post,put*** et ***delete.***

Notre fonction (deuxième paramètre) peut aussi afficher une chaine de caractères comme suit :



…



On peut même ajouter du HTML dans le texte :

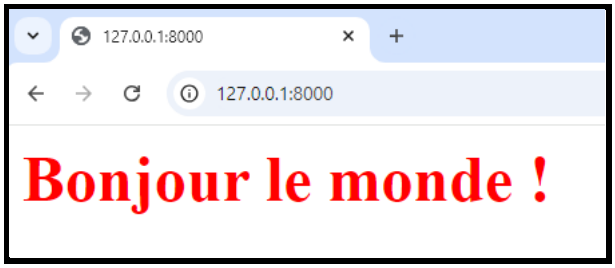
```php
<?php

use Illuminate\Support\Facades\Route;

// Route::get('/', function () {
//     return view('welcome');
// });

Route::get('/', function(){
    return "<h1><font color='red'>Bonjour le monde !</font></h1>";
});
```

Résultat :



Maintenant changeons la route :

```php
<?php

use Illuminate\Support\Facades\Route;

// Route::get('/', function () {
//     return view('welcome');
// });

// Route::get('/', function(){
//     return "<h1><font color='red'>Bonjour le monde !</font></h1>";
// });

Route::get('/home', function(){
    return "<h1><font color='red'>Bonjour le monde !</font></h1>";
});
```

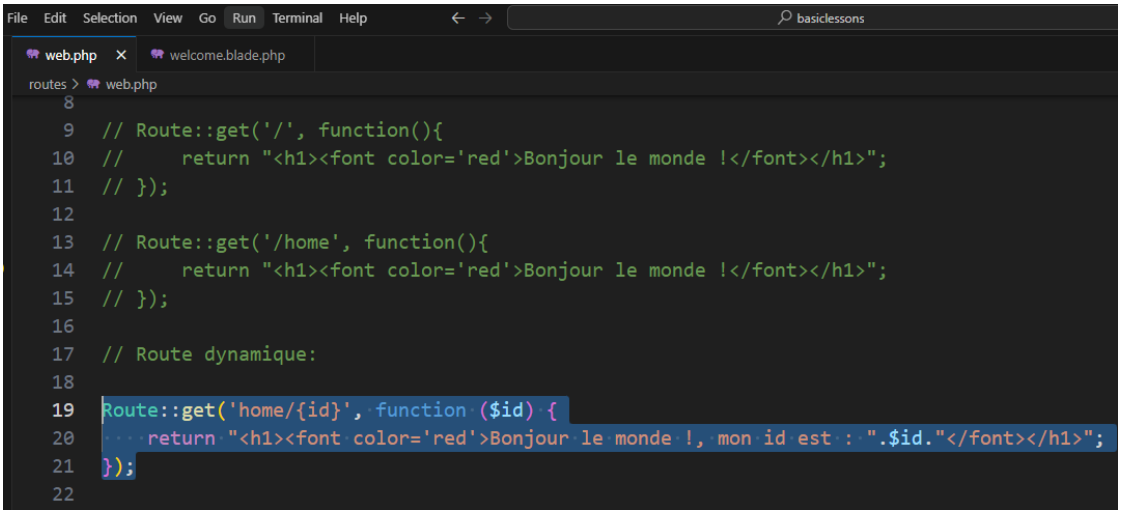
Actualisons le navigateur :



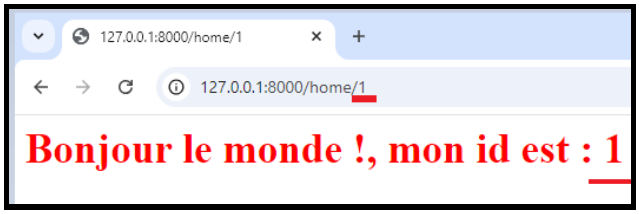
Mais : /home



Cas de route dynamique :



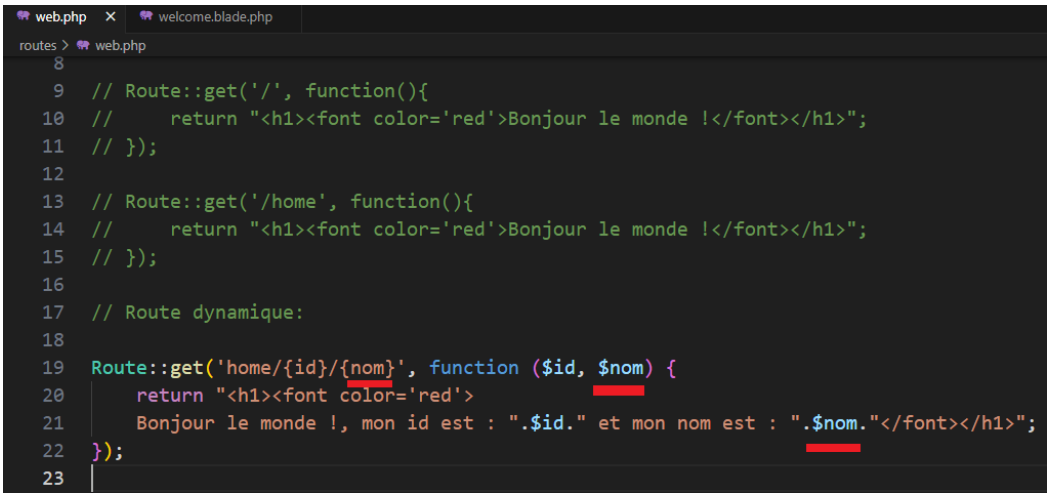
Il suffit de saisir des valeurs dans l'url pour les récupérer et les afficher dans la vue :



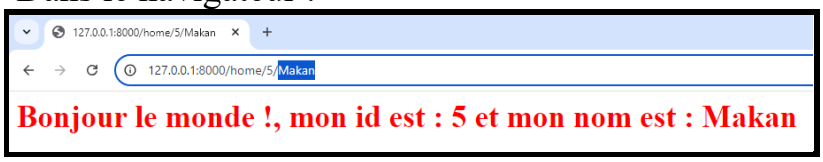
Encore :



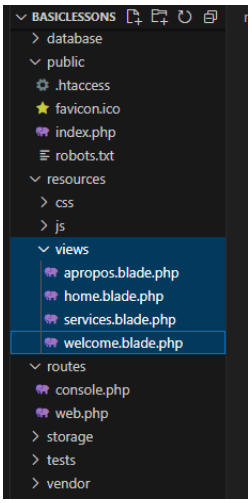
On peut aller plus loin encore :



-Dans le navigateur :



-Dans le dossier **views**, créer les fichiers **home.blade.php**, **apropos.blade.php** et **services.blade.php:**



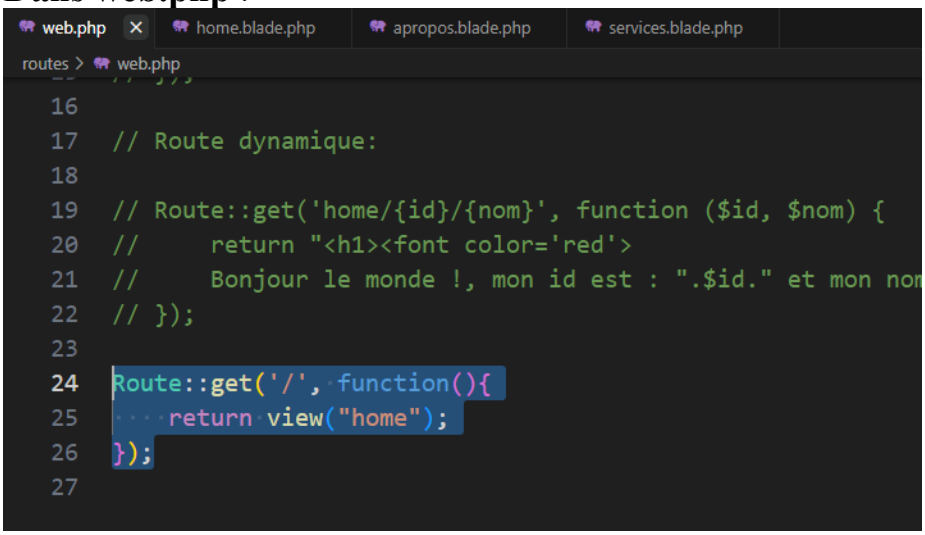
Le formateur vous remet 3 fichiers : index.html, apropos.html et services.html.

5. Route (PARTIE 3)

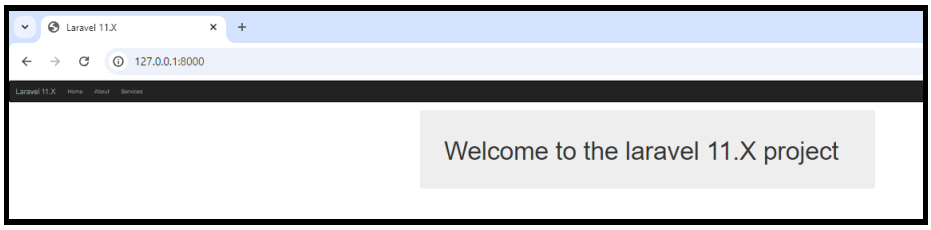
Une copier le contenu des 3 fichiers dans les fichiers respectifs, on va voir comment les afficher dans la vue.

Construction des routes.

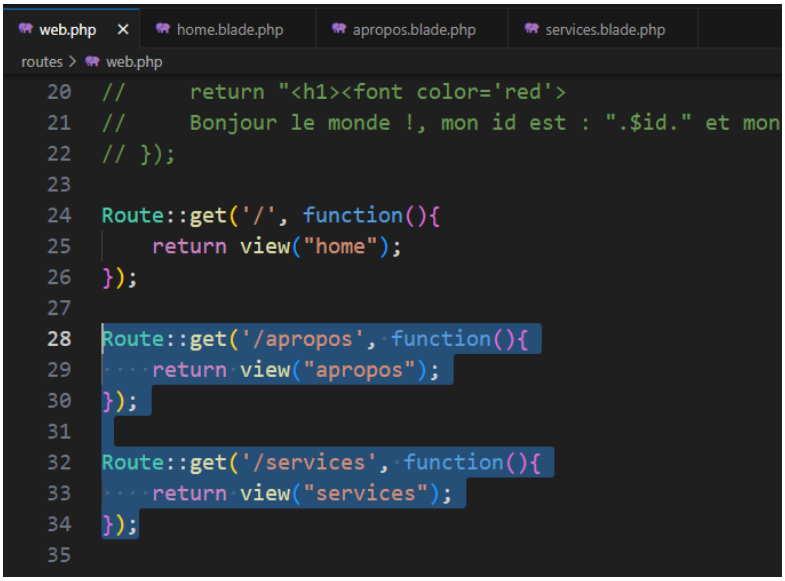
Dans web.php :



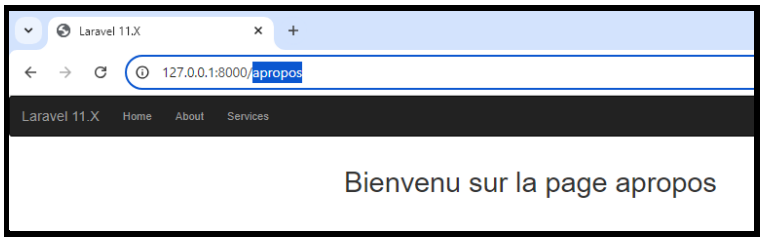
Résultat :



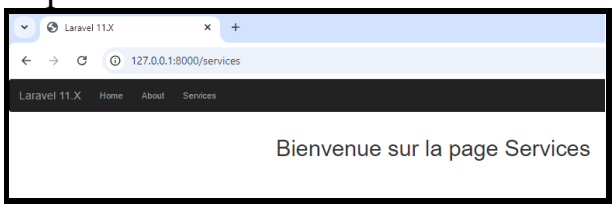
Pour les deux autres fichiers :



Résultat :

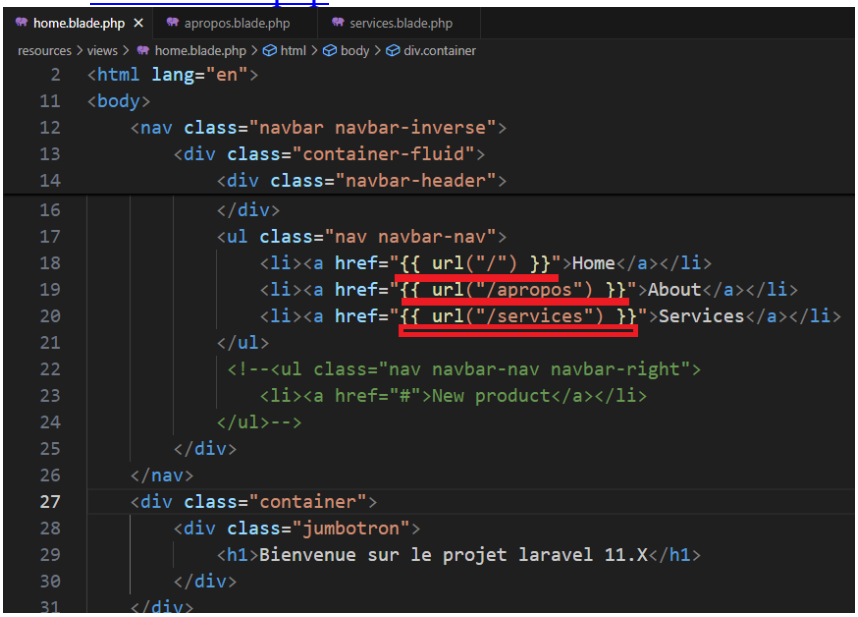


Et pour services :

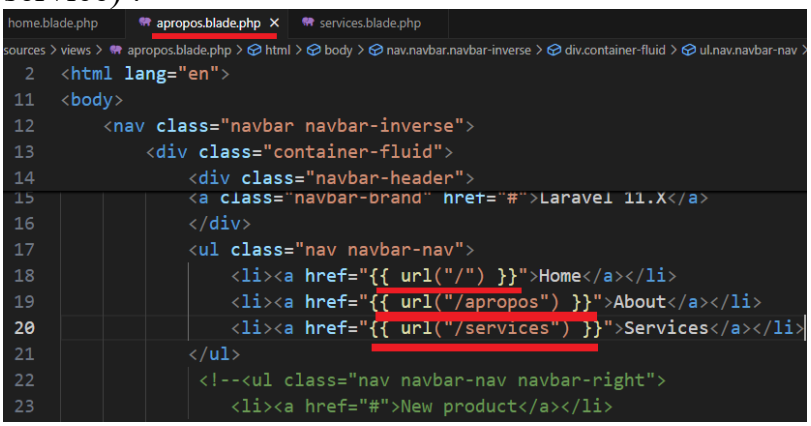


Le menu est inactif pour le moment.

Dans home.blade.php



On copie le code souligné et le place dans les deux autres fichiers (apropos et service) :



```
home.blade.php  apropos.blade.php  services.blade.php
resources > views > services.blade.php > html > body > div.container > h1
 2  <html lang="en">
 3  <head>
10  </head>
11  <body>
12      <nav class="navbar navbar-inverse">
13          <div class="container-fluid">
14              <div class="navbar-header">
15              <a class="navbar-brand" href="#">Laravel 11.X</a>
16              </div>
17              <ul class="nav navbar-nav">
18                  <li><a href="{{ url("/") }}">Home</a></li>
19                  <li><a href="{{ url("/apropos") }}">About</a></li>
20                  <li><a href="{{ url("/services") }}">Services</a></li>
21              <!--<ul class="nav navbar-nav navbar-right">
22                  <li><a href="#">New product</a></li>
23              </ul>-->
24          </div>
25      </nav>
```

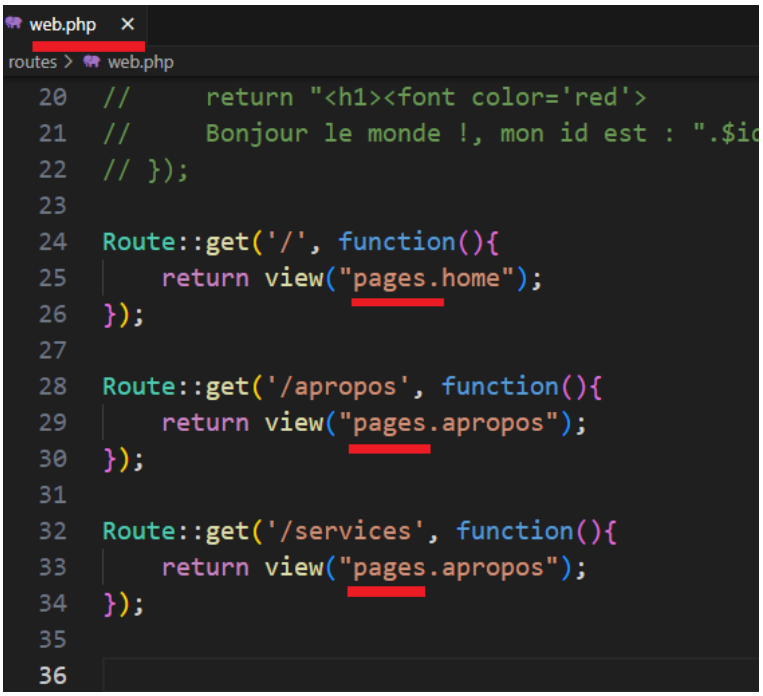
A partir de maintenant, le menu devient actif. Un clic sur chaque menu, affiche la vue correspondante.

Une autre méthode de notation :

```
home.blade.php  apropos.blade.php  services.blade.php
resources > views > services.blade.php > html > body > nav.navbar.navbar-inverse > div.container-fluid
 2  <html lang="en">
11  <body>
12      <nav class="navbar navbar-inverse">
13          <div class="container-fluid">
14              <div class="navbar-header">
15              <a class="navbar-brand" href="#">Laravel 11.X</a>
16              </div>
17              <ul class="nav navbar-nav">
18                  <li><a href="{{ URL:: to("/") }}">Home</a></li>
19                  <li><a href="{{ URL:: to("/apropos") }}">About</a></li>
20                  <li><a href="{{ URL:: to("/services") }}">Services</a></li>
21              <!--<ul class="nav navbar-nav navbar-right">
22                  <li><a href="#">New product</a></li>
23              </ul>-->
24          </div>
25      </nav>
```

Organisation des fichiers dans le dossier views :

- Créez un dossier pages dans **views** et glisser les 3 fichiers (home, apropos et services) dans ce dernier.
- Allez changer les chemins des fichiers (vues) à afficher dans web.php pour prendre en compte le dossier pages comme ci-dessous :



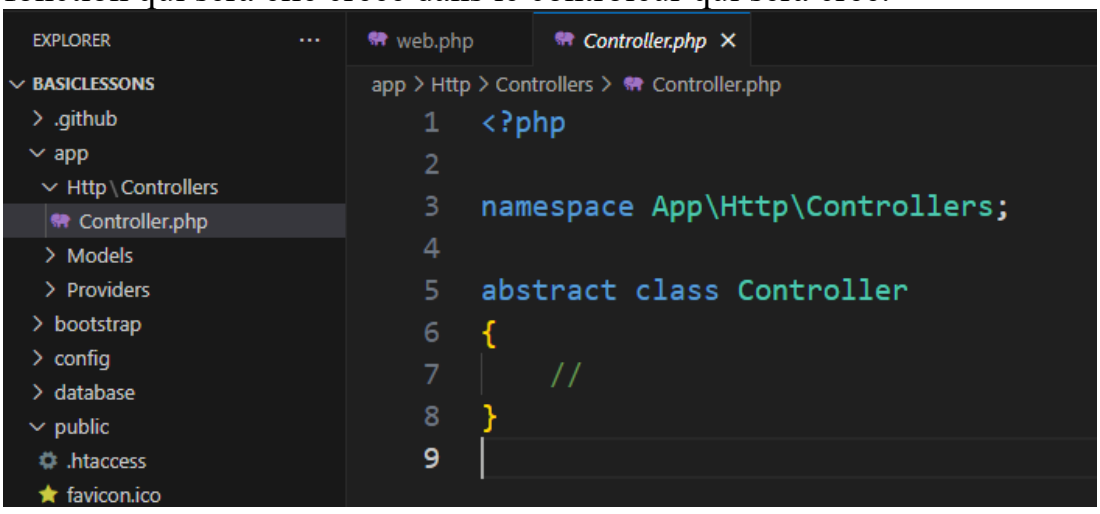
NB : mettre service à la place de apropos : ligne 33

On teste et tout fonctionne correctement.

6. Contrôleur (PARTIE 1)

La notion de Controller essaye d'alléger le fichier **web.php** en matière de code. Elle demande de supprimer le code de la méthode qui est en deuxième argument de la méthode **get( )** et de le mettre ailleurs dans un fichier Controller qu'il faut créer.

A la place du deuxième paramètre, nous allons placer une référence à une fonction qui sera elle créée dans le contrôleur qui sera créé.



Le contrôleur ci-dessus est le contrôleur par défaut que nous n'allons pas utiliser. AU contraire, nous allons créer nos propres contrôleurs en ligne de commande.

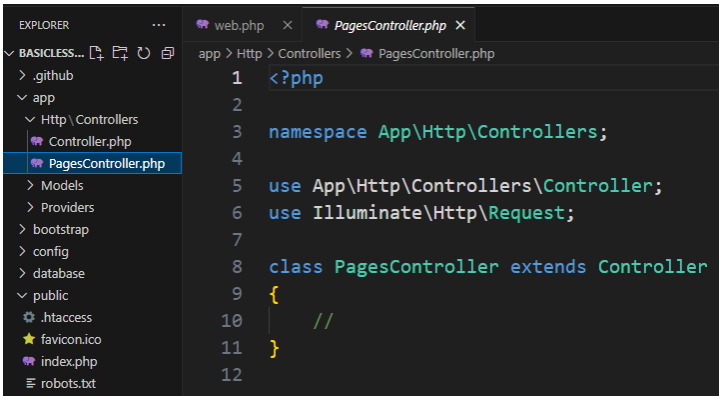
Création de notre contrôleur : PagesController

```
C:\xampp\htdocs\basiclessons>php artisan make:controller PagesController

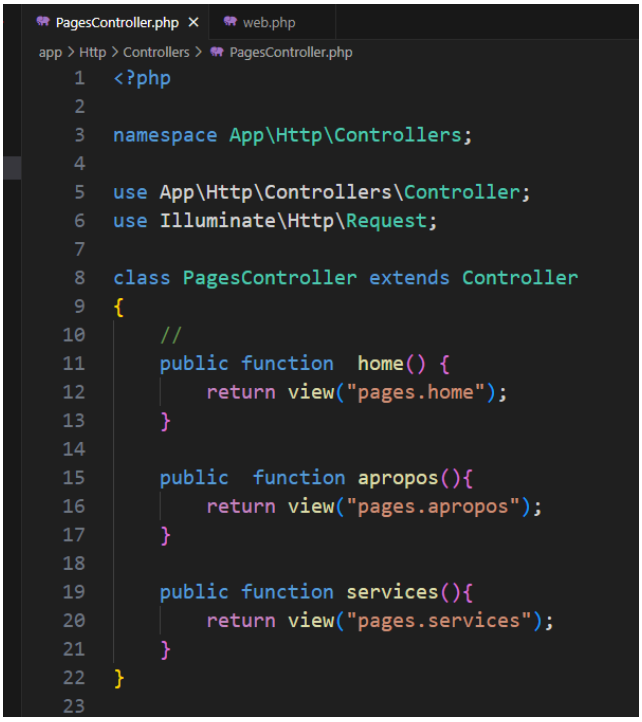
   INFO  Controller [C:\xampp\htdocs\basiclessons\app\Http\Controllers\PagesController.php] created successfully.


C:\xampp\htdocs\basiclessons>
```

Vérification :

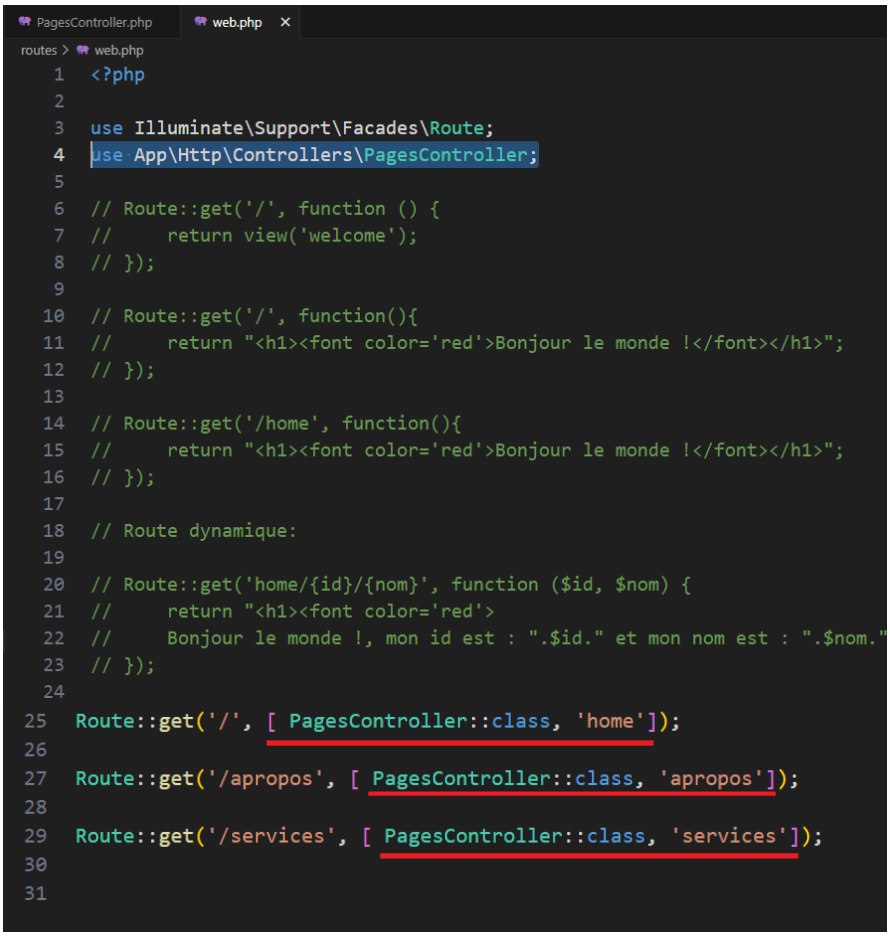


Ensuite :



7. Contrôleur (PARTIE 2)

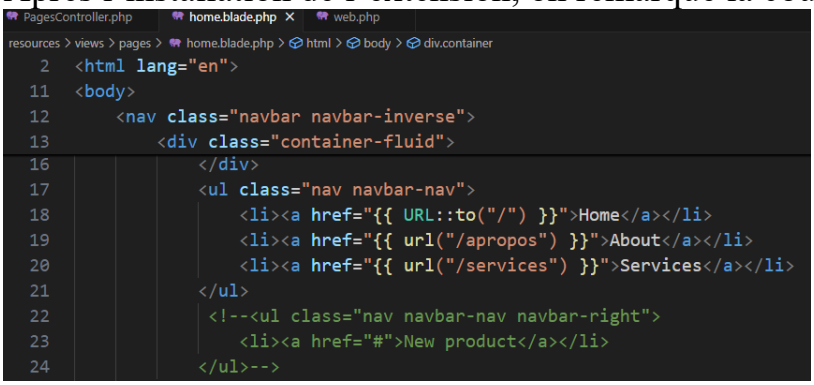
Et **web.php** devient alors :



8. Bladetemplating (PARTIE 1)

Installer l'extension : **Laravel BladeSnippets** : pour la colorisation

Après l'installation de l'extension, on remarque la couleur :



On remarque que dans les trois fichiers (**home, apropos** et **services**) un code similaire que nous allons essayer de le factoriser.

-Dans le dossier **views**, créer un dossier **layouts** et dans ce dernier créer un fichier appelé **master.blade.php.**

Copier le code commençant de DOCTYPE jusqu'à **container** et le coller dans le fichier ***master.blade.php***

Voici **master.blade.php**

```
home.blade.php  apropos.blade.php  services.blade.php  master.blade.php
resources > views > layouts > master.blade.php > html > body > div.container
 2  <html lang="en">
11  <body>
12      <nav class="navbar navbar-inverse">
13          <div class="container-fluid">
17              <ul class="nav navbar-nav">
21              </ul>
22               <!--<ul class="nav navbar-nav navbar-right">
23                  <li><a href="#">New product</a></li>
24              </ul>-->
25          </div>
26      </nav>
27      <div class="container">
28      {{--  Début du contenu  --}}
29
30      {{--  Fin du contenu  --}}
31      </div>
32  </body>
33  </html>
34
```

[home.blade.php](#)

```
home.blade.php  apropos.blade.php  services.blade.php  master.blade.php
resources > views > pages > home.blade.php > ...
1
2
3           <div class="jumbotron">
4               <h1>Bienvenue sur le projet laravel 11.X</h1>
5           </div>
6
```

apropos.blade.php

```
home.blade.php  apropos.blade.php  services.blade.php  master.blade.php
resources > views > pages > apropos.blade.php > ...
1
2           <h1>Bienvenu sur la page apropos</h1>
3
```

services.blade.php

```
home.blade.php  apropos.blade.php  services.blade.php  master.blade.php
resources > views > pages > services.blade.php > ...
1
2           <h1>Bienvenue sur la page Services</h1>
3
```

9. Blade templating (PARTIE 2)

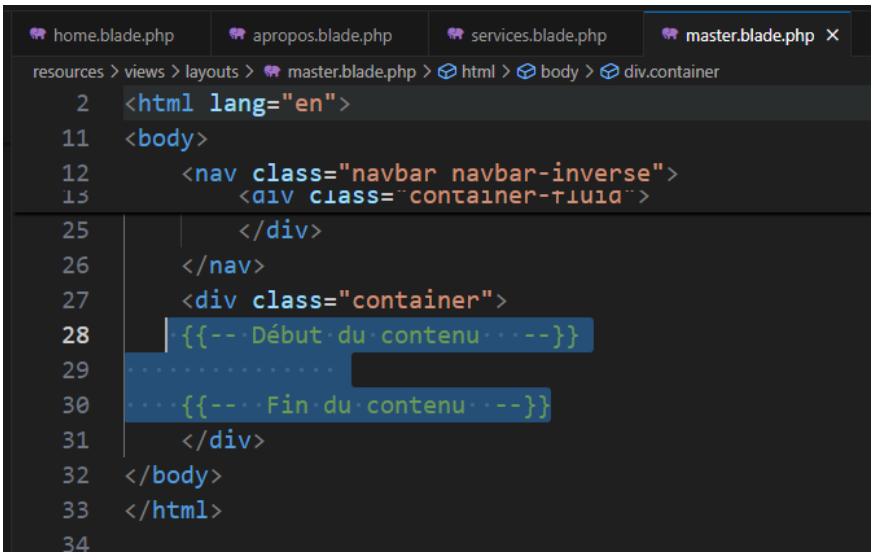
Aucune barre de navigation dans mes pages :







Maintenant, ***master.blade.php*** contient tout le code similaire des trois autres fichiers et il faut le rendre dynamique.
C'est-à-dire quand on clique sur un menu, son contenu doit être chargé dans la partie en surbrillance de **master.blade.php**



Pour faire cela, il faut faire hériter ***master.blade.php*** par tous les autres fichiers comme suit :

home.blade.php

```blade
@extends('layouts.master')

@section('contenu')

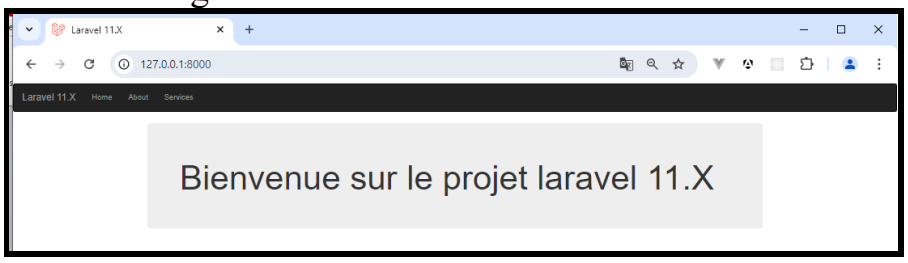
    <div class="jumbotron">
        <h1>Bienvenue sur le projet laravel 11.X</h1>
    </div>

@endsection
```

master.blade.php

```blade
<html lang="en">
<body>
    <nav class="navbar navbar-inverse">
        <div class="container-fluid">
        </div>
    </nav>
    <div class="container">
        {{-- Début du contenu   --}}
            @yield('contenu')
        {{--  Fin du contenu  --}}
    </div>
</body>
</html>
```

Dans le navigateur :



Le menu est bien affiché. Donc le contenu a été bien chargé.

Par contre quand on clique sur *apropos :* le menu est absent.

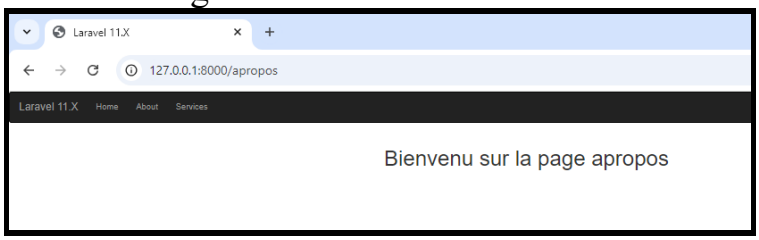


On fait la même manipulation pour les autres fichiers :

apropos.blade.php

```
home.blade.php | apropos.blade.php | services.blade.php | master.blade.php
resources > views > pages > apropos.blade.php > ...
1  @extends('layouts.master')
2
3      @section('contenu')
4
5       <h1>Bienvenu sur la page apropos</h1>
6      @endsection
```

Dans le navigateur :



services.blade.php

```
home.blade.php | apropos.blade.php | services.blade.php | master.blade.php
resources > views > pages > services.blade.php > ...
1  @extends('layouts.master')
2
3  @section('contenu')
4      <h1>Bienvenue sur la page Services</h1>
5  @endsection
6
7
```

Dans le navigateur :



A propos de l'include :

- Créons un fichier **navbar.blade.ph** dans le dossier **layouts.**
- Dans le fichier ***master.blade.php***, coupez le code de la partie **nav** et le coller dans **navbar.blade.php**

navbar.blade.php

```
home.blade.php | apropos.blade.php | services.blade.php | navbar.blade.php | master.blade.php
resources > views > layouts > navbar.blade.php > ...
<nav class="navbar navbar-inverse">
    <div class="container-fluid">
        <div class="navbar-header">
        <a class="navbar-brand" href="#">Laravel 11.X</a>
        </div>
        <ul class="nav navbar-nav">
            <li><a href="{{ URL::to("/") }}">Home</a></li>
            <li><a href="{{ url("/apropos") }}">About</a></li>
            <li><a href="{{ url("/services") }}">Services</a></li>
        </ul>
         <!--<ul class="nav navbar-nav navbar-right">
            <li><a href="#">New product</a></li>
        </ul>-->
    </div>
</nav>
```

master.blade.php

```
navbar.blade.php | master.blade.php
resources > views > layouts > master.blade.php > html > body
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width,
  <link rel="stylesheet" href="https://maxcdn.bootst
  <script src="https://ajax.googleapis.com/ajax/libs
  <script src="https://maxcdn.bootstrapcdn.com/boots
</head>
<body>

  @include('layouts.navbar')

    <div class="container">
        {{-- Début du contenu   --}}
            @yield('contenu')
        {{--  Fin du contenu  --}}
    </div>
</body>
</html>
```

Dans le navigateur :



Tout fonctionne correctement.

On doit rendre également le titre dynamique :

master.blade.php | home.blade.php × | apropos.blade.php | services.blade.php

resources > views > pages > home.blade.php > ...

```
1  @extends('layouts.master')
2
3  @section('title')
4      Home
5  @endsection
6
7  @section('contenu')
8      <div class="jumbotron">
9          <h1>Bienvenue sur le projet laravel 11.X</h1>
10     </div>
11 @endsection
12
```

Et

master.blade.php | apropos.blade.php × | services.blade.php

resources > views > pages > apropos.blade.php > ...

```
1  @extends('layouts.master')
2
3
4  @section('title')
5      Apropos
6  @endsection
7
8  @section('contenu')
9
10     <h1>Bienvenu sur la page apropos</h1>
11 @endsection
12
```

Et

master.blade.php | apropos.blade.php | services.blade.php ×

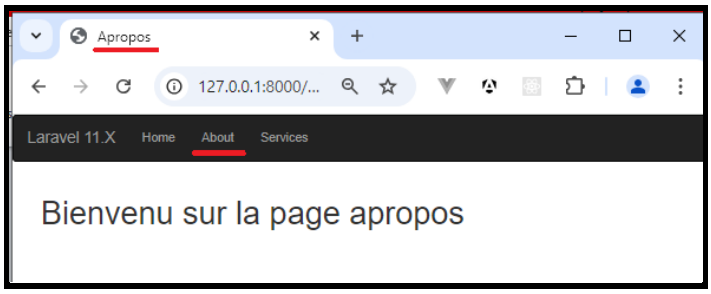
resources > views > pages > services.blade.php > ...

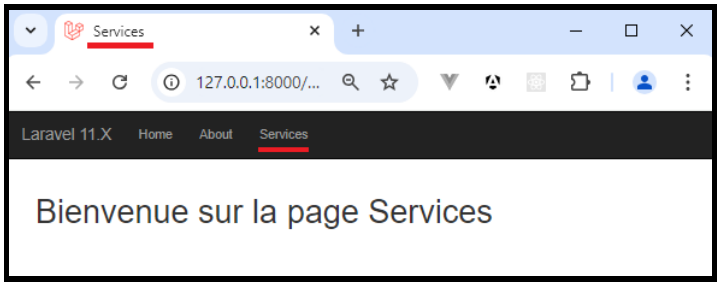
```
1  @extends('layouts.master')
2
3  @section('title')
4      Services
5  @endsection
6
7  @section('contenu')
8      <h1>Bienvenue sur la page Services</h1>
9  @endsection
10
```

ET

Dans le navigateur :

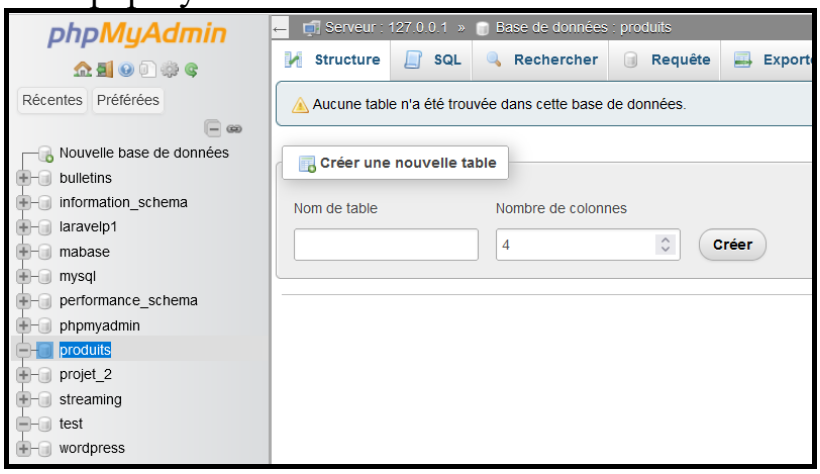






10. Modèles et Migrations (PARTIE 1)

Les modèles représentent les différentes tables que l'on désire créer dans notre base de données.
Dès qu'un modèle est créé, son fichier est stocké dans le dossier **migrations** lui aussi contenu dans **database.**
Dans phpMyAdmin nous allons créer une bade de données : **produits**



Notre base de données est créée mais ne contient aucune table pour le moment.
Les tables doivent être créées dans l'interface laravel.

Nous allons créer notre modèle Product (au singulier et la première lettre en majuscule) c'est ne classe qui sera stockée dans le dossier **Models** dans le dossier App.
Un fichier de migration correspondant sera aussi créé dans le dossier **migrations**
Après migration, une table products (au pluriel) sera créée dans la base de données : **produits**.

```
C:\xampp\htdocs\basiclessons>php artisan make:model Product -m

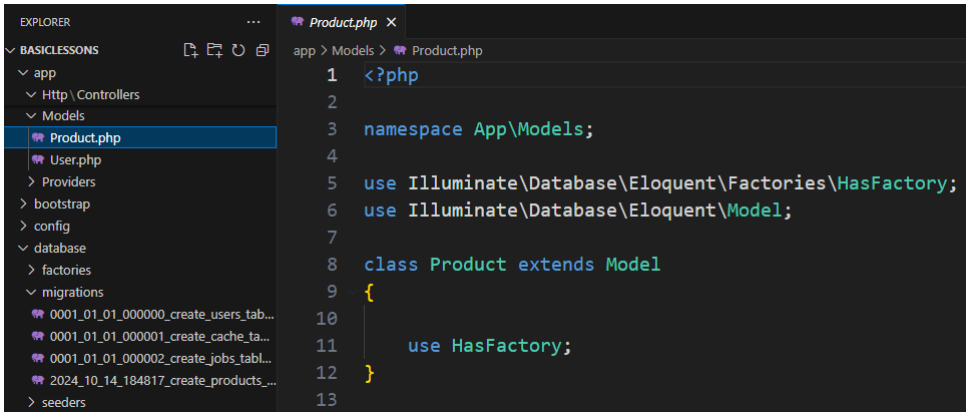
   INFO  Model [C:\xampp\htdocs\basiclessons\app\Models\Product.php] created successfully.

   INFO  Migration [C:\xampp\htdocs\basiclessons\database\migrations/2024_10_14_184817_create_products_table.php] created successfully.

C:\xampp\htdocs\basiclessons>
```

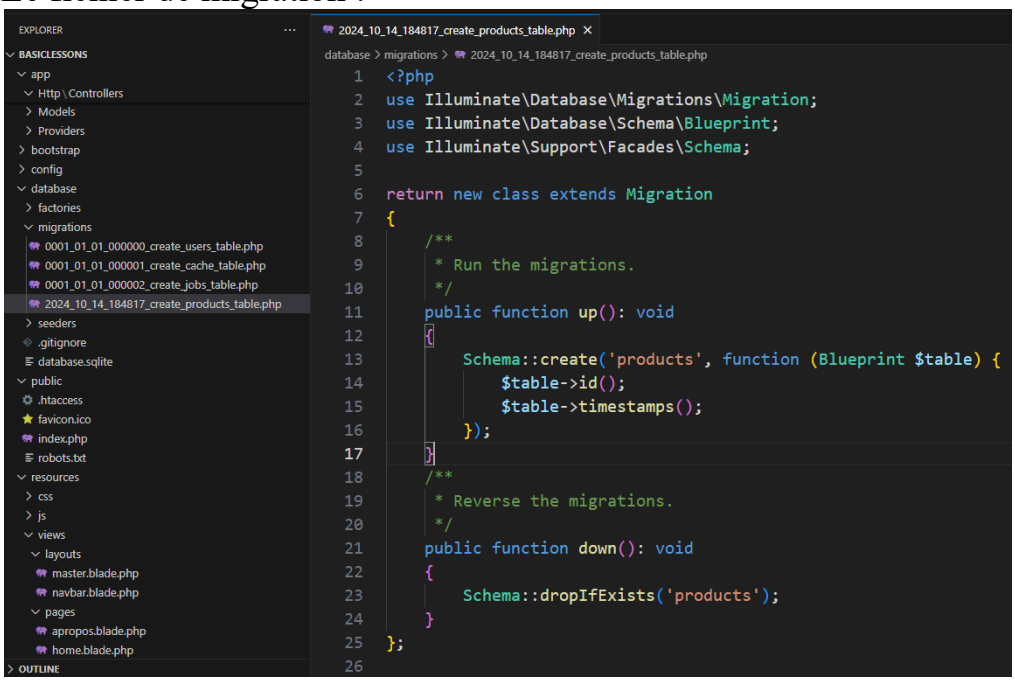
Résultat :

La classe **Product**



11. Modèles et Migrations (PARTIE 2)

Le fichier de migration :



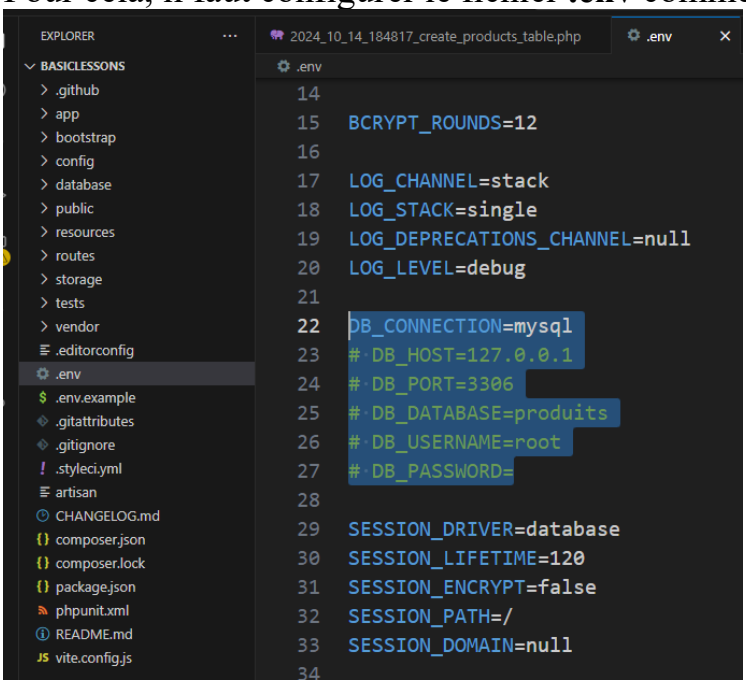
Maintenant, il faut personnaliser le fichier ci-dessus qui représente notre table en base une fois la migration lancée.

Ajoutons les différents champs dans notre table :

```php
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('products', function (Blueprint $table) {
            $table->id();
            $table->string('product_name');
            $table->nteger('product_price');
            $table->mediumText('product_description');
            $table-> timestamps();
        });
    }
    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('products');
    }};
```

Maintenant il reste à faire la connexion entre l'application et la base de données. Pour cela, il faut configurer le fichier **.env** comme ci-dessous :



- Lancer la migration en base :

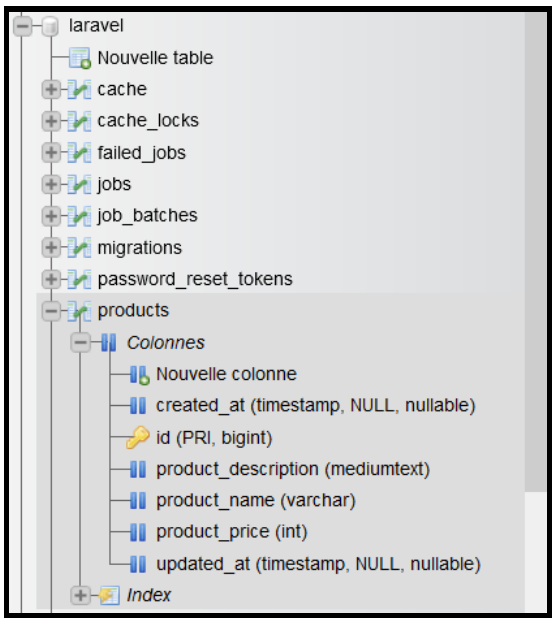
```
C:\xampp\htdocs\basiclessons>php artisan migrate

   INFO  Running migrations.

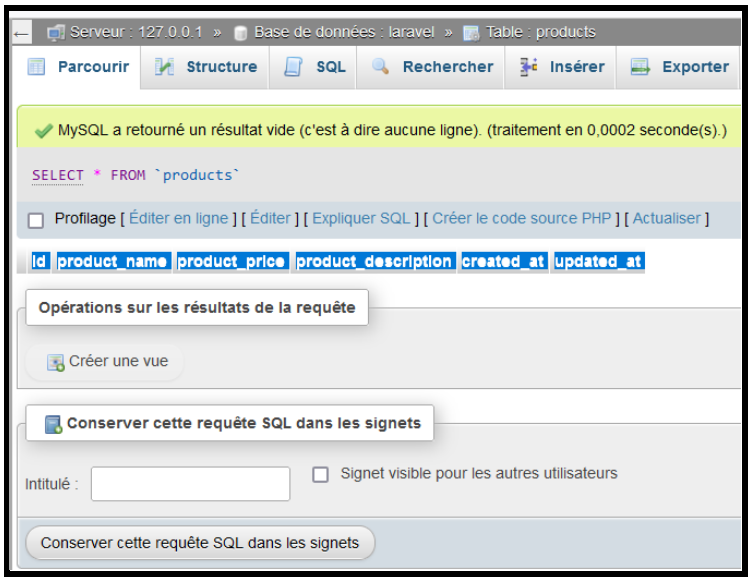
  2024_10_14_184817_create_products_table .............

C:\xampp\htdocs\basiclessons>
```

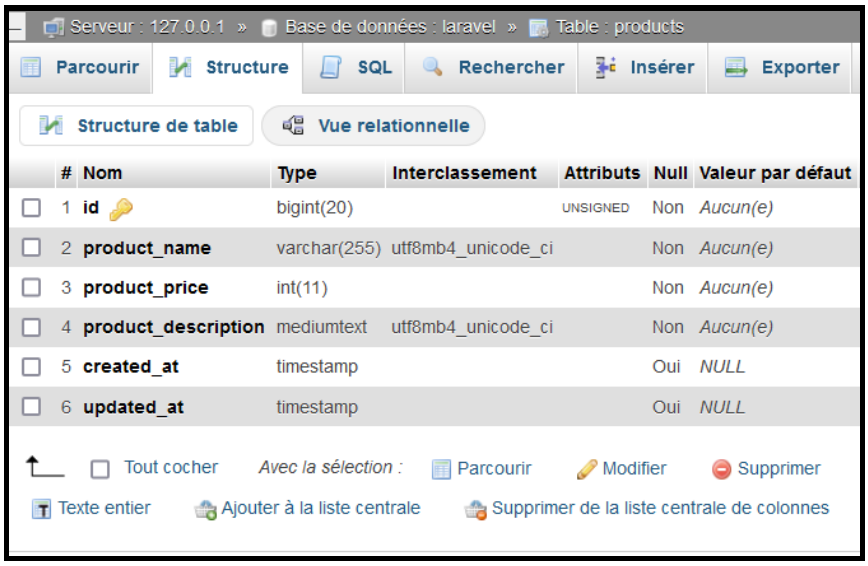
Résultat :



Et :



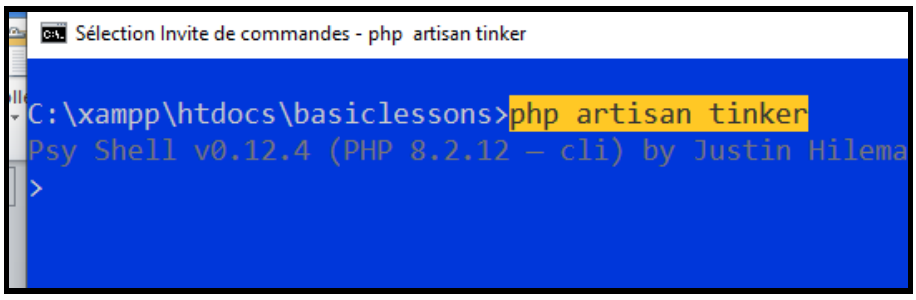
Structure



Comment interagir avec le modèle ?

12. Enregistrement de données (PARTIE 1) via le terminal

Lancer la commande suivante :



- Affichons le nombre d'éléments dans Product :

```
> App\Models\Product::count()
= 0
```

Nous allons instancier notre classe Product :

```
> $produit = new App\Models\Product
=                   {#5186}

> $produit->product_name = "Pomme"
= "Pomme"

> $produit->product_price = 100
= 100

> $produit->product_description = "La meilleur de la récolte"
= "La meilleur de la récolte"

>
```

Maintenant il faudra sauvegarder l'objet $produit dans la base à l'aide de la méthode save :

```
> $produit->save()
= true

>
```

Dans phpMyAdmin :



Insérons encore un autre produit :

```
> $produit->product_name = "Mangue"
= "Mangue"

> $produit->product_price = 200
= 200

> $produit->product_description = "La mangue ddu jardin"
= "La mangue ddu jardin"

> $produit->save()
= true
```

Résultat :



On remarque le premier enregistrement est écrasé par le nouveau enregistrement.
Qu'est-ce qu'il faut faire alors ?
Il faudra ré-instancier encore la classe :

```
> $produit = new App\Models\Product
=                   {#5185}

> $produit->product_name = "Pomme"
= "Pomme"

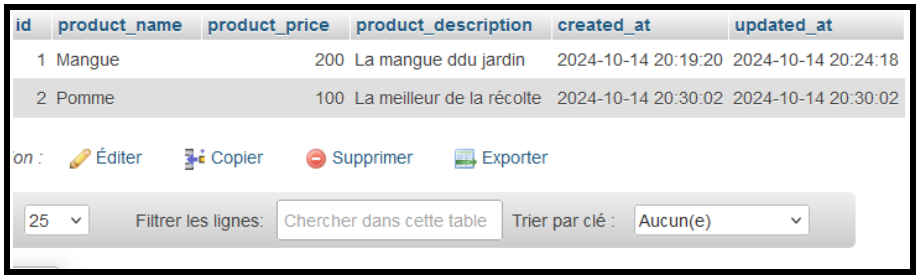
> $produit->product_price = 100
= 100

> $produit->product_description = "La meilleur de la récolte"
= "La meilleur de la récolte"

> $produit->save()
= true

>
```

Résultat :



id	product_name	product_price	product_description	created_at	updated_at
1	Mangue	200	La mangue ddu jardin	2024-10-14 20:19:20	2024-10-14 20:24:18
2	Pomme	100	La meilleur de la récolte	2024-10-14 20:30:02	2024-10-14 20:30:02

on : Éditer Copier Supprimer Exporter

25 Filtrer les lignes: Chercher dans cette table Trier par clé : Aucun(e)

Les deux insertions sont maintenant présentes dans la base de données.

13. Affichage de données (PARTIE 1)

Affichons maintenant les produits contenus dans la base de données dans la page Services.

Première méthode : Laravel **QueryBuilder**

-Dans le contrôleur : PagesController

PagesController.php

app > Http > Controllers > PagesController.php

```php
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;

class PagesController extends Controller
{
    //
    public function  home() {
        return view("pages.home");
    }
    public  function apropos(){
        return view("pages.apropos");
    }

    public function services(){
        $produits = DB::table('products')->get();
        return view("pages.services")->with('produits', $produits);
    }
}
```

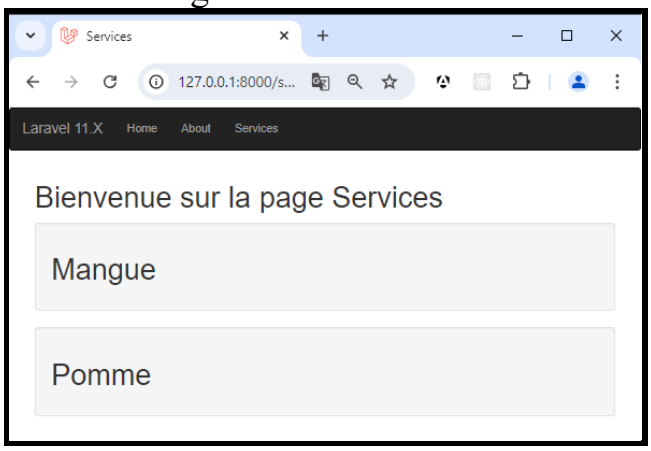
-Dans **services.blade.php**

PagesController.php | services.blade.php

resources > views > pages > services.blade.php > ...

```
1  @extends('layouts.master')
2
3  @section('title')
4      Services
5  @endsection
6
7  @section('contenu')
8
9      <h1>Bienvenue sur la page Services</h1>
10     @foreach ($produits as $produit)
11         <div class="well">
12             <h1>{{ $produit->product_name }}</h1>
13         </div>
14     @endforeach
15 @endsection
16
```

Dans le navigateur :



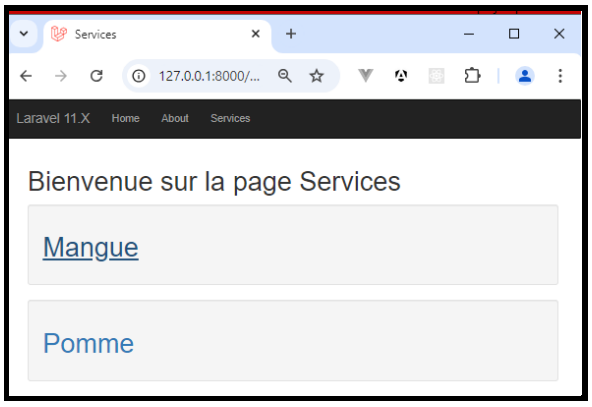
-Mettre en lien sur les noms :

PagesController.php | services.blade.php

resources > views > pages > services.blade.php > ...

```
1  @extends('layouts.master')
2
3  @section('title')
4      Services
5  @endsection
6
7  @section('contenu')
8
9      <h1>Bienvenue sur la page Services</h1>
10     @foreach ($produits as $produit)
11         <div class="well">
12             <h1><a href="#">{{ $produit->product_name }}</a></h1>
13         </div>
14     @endforeach
15 @endsection
16
```

Résultat :



14. Affichage de données (PARTIE 2)

Affichage des produits par nom selon l'ordre croissant

-Dans PagesController :

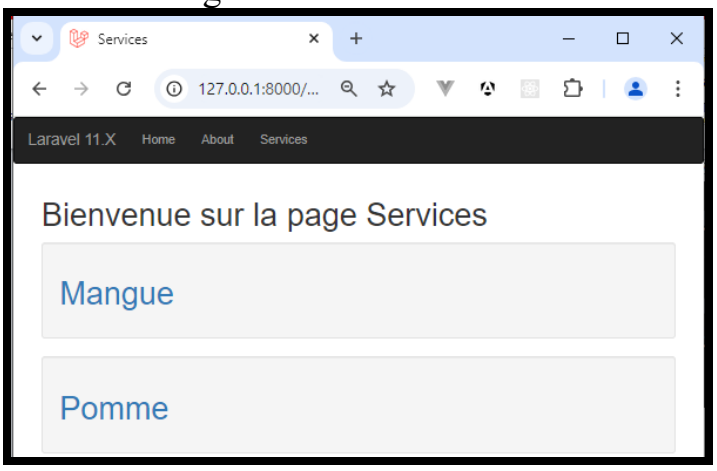
```php
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;

class PagesController extends Controller
{
    //
    public function  home() {
        return view("pages.home");
    }
    public  function apropos(){
        return view("pages.apropos");
    }
    public function services(){
        $produits = DB::table('products')->orderBy('product_name', 'asc')->get();
        return view("pages.services")->with('produits', $produits);
    }
}
```

-Dans le navigateur :



Les produits sont affichés par ordre croissant.

Affichage par ordre décroissant :

Dans **PagesController**, il suffit de remplacer *asc* par *desc*

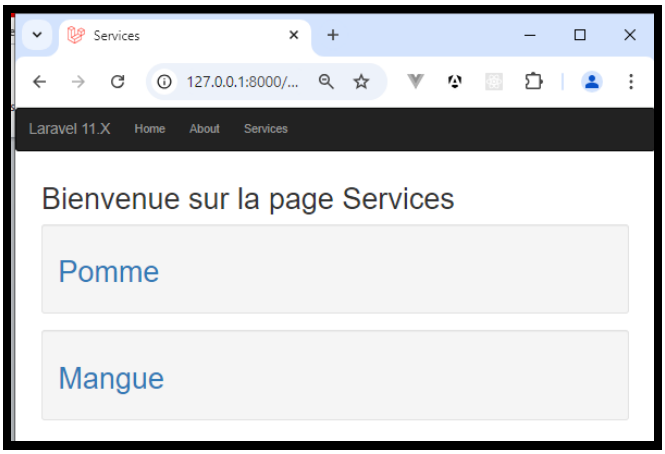
```php
<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;

class PagesController extends Controller
{
    //
    public function  home() {
        return view("pages.home");
    }
    public  function apropos(){
        return view("pages.apropos");
    }
    public function services(){
        $produits = DB::table('products')->orderBy('product_name', 'desc')->get();
        return view("pages.services")->with('produits', $produits);
    }
}
```

Résultat :

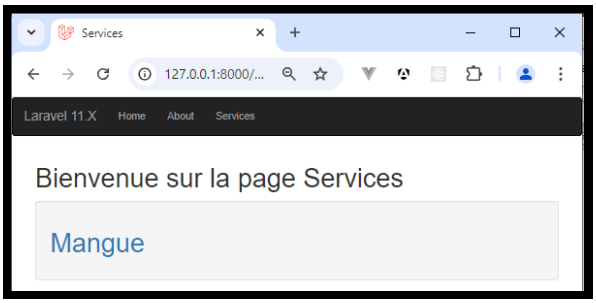


Les produits sont affichés par ordre décroissants.

Affichage par page en fonction du nombre de produits par page qu'il faut donner en argument de la méthode : paginate(x) :

```php
class PagesController extends Controller
{
    //
    public function  home() {
        return view("pages.home");
    }
    public  function apropos(){
        return view("pages.apropos");
    }
    public function services(){
        $produits = DB::table('products')->paginate(1);
        return view("pages.services")->with('produits', $produits);
    }
}
```

Résultat :

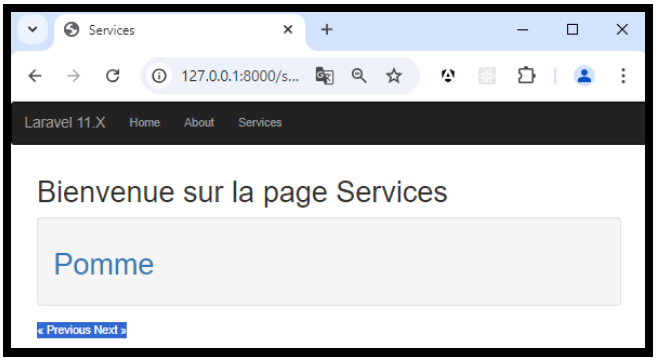


Affichage de la pagination en dessous de la page permettant de passer d'une page à une autre :

-Dans **service.blade.php** :

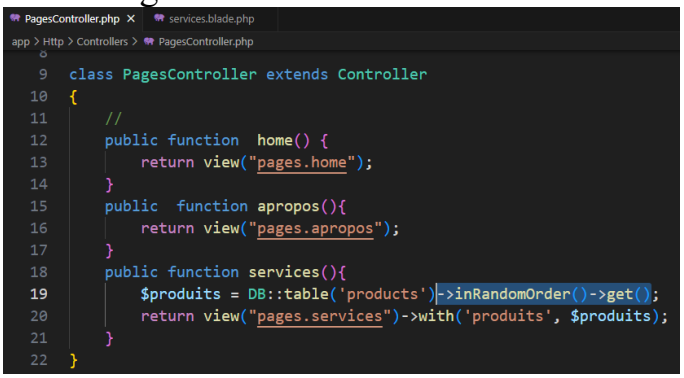


Résultat :



Affichage par ordre aléatoire : inRandomOrder( )

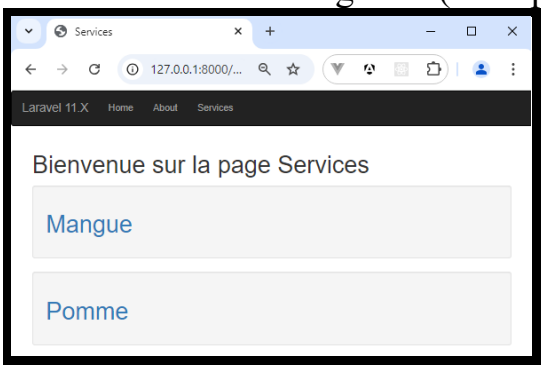
-Dans PagesController



Dans ce cas, il faudra commenter la ligne 15 dans service.blade.php

```blade
    <h1>Bienvenue sur la page Services</h1>
    @foreach ($produits as $produit)
        <div class="well">
            <h1><a href="#">{{ $produit->product_name }}</a></h1>
        </div>
    @endforeach
    {{-- {{ $produits->links() }} --}}
@endsection
```

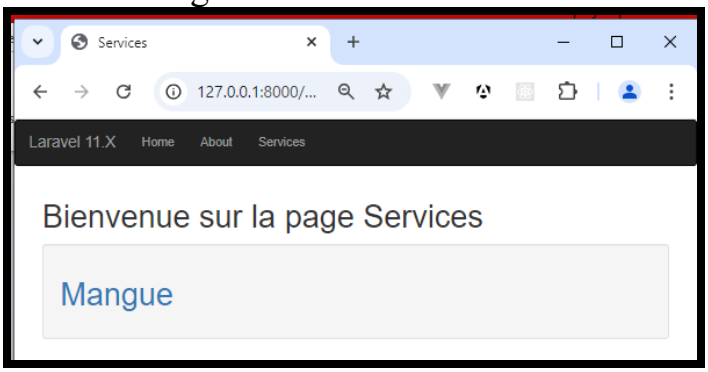
Résultat dans le navigateur : l'affichage est aléatoire au fur et à mesure de l'actualisation du navigateur (de la page) :



Affichage avec limitation : affichons par 1 produit :
-Dans PagesController :

```php
{
    //
    public function  home() {
        return view("pages.home");
    }
    public  function apropos(){
        return view("pages.apropos");
    }
    public function services(){
        $produits = DB::table('products')->inRandomOrder()->limit(1)->get();
        return view("pages.services")->with('produits', $produits);
    }
}
```

Dans le navigateur :



La méthode **QueryBuilder** n'est pas assez utilisée. La méthode des modules est beaucoup plus utilisée et c'est cette dernière qui nous allons utiliser dans tout le reste du cours.

Utilisation des Modèles : Eloquent

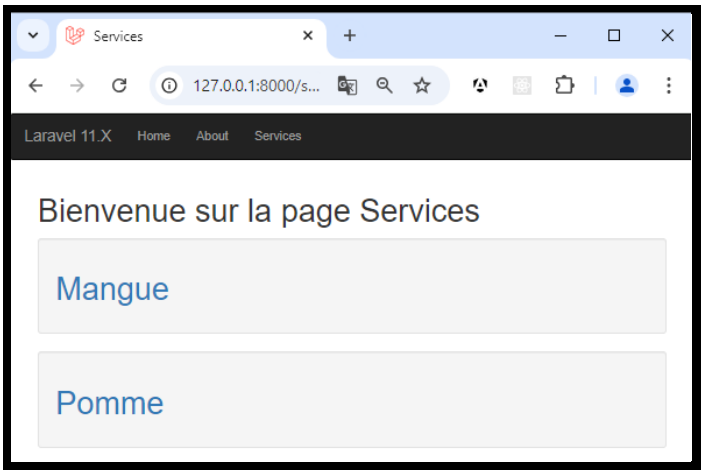
-Dans PagesController :

```php
use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;
use App\Models\Product;

class PagesController extends Controller
{
    //
    public function  home() {
        return view("pages.home");
    }
    public  function apropos(){
        return view("pages.apropos");
    }
    public function services(){
        // $produits = DB::table('products')->inRandomOrder()->limit(1)->get();
        $produits = Product::get();

        return view("pages.services")->with('produits', $produits);
    }
}
```

Résultat :



On peut aussi utiliser la méthode : All( ) :

```php
{
    public function  home() {
        return view("pages.home");
    }
    public  function apropos(){
        return view("pages.apropos");
    }
    public function services(){
        // $produits = DB::table('products')->inRandomOrder()->limit(1)->get();
        $produits = Product::all();

        return view("pages.services")->with('produits', $produits);
    }
}
```

On obtient le même résultat à l'affichage dans le navigateur.

Utiliser encore la méthode ***paginate( )***

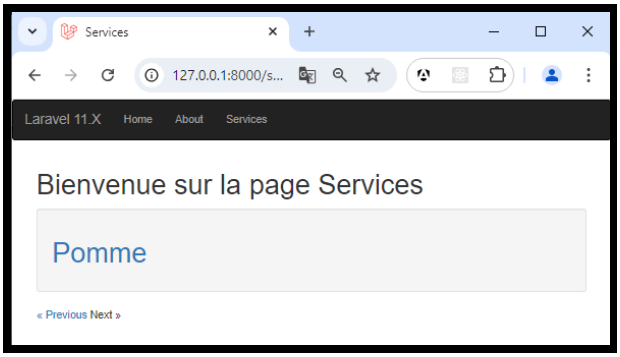
```php
{
    public function  home() {
        return view("pages.home");
    }
    public  function apropos(){
        return view("pages.apropos");
    }
    public function services(){
        // $produits = DB::table('products')->inRandomOrder()->limit(1)->get();
        $produits = Product::paginate(1);

        return view("pages.services")->with('produits', $produits);
    }
}
```

-Dans service.blade.php, il faudra décommenter la ligne en commentaire :

```blade
    <h1>Bienvenue sur la page Services</h1>
    @foreach ($produits as $produit)
        <div class="well">
            <h1><a href="#">{{ $produit->product_name }}</a></h1>
        </div>
    @endforeach
    {{ $produits->links() }}
@endsection
```

Résultat :



Autre utilisation :

-Dans PagesController :

a- Ordre croissant :

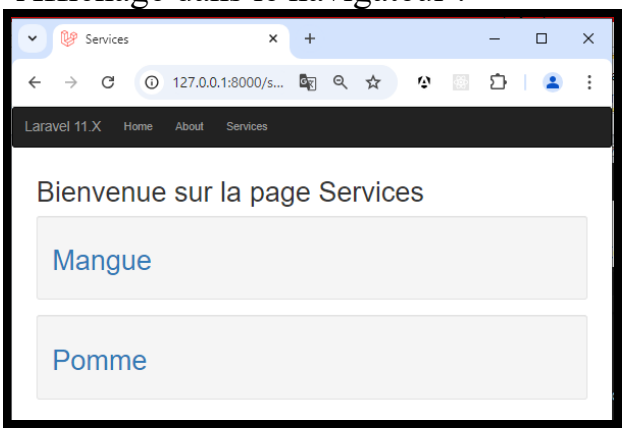
```php
{
    public function  home() {
        return view("pages.home");
    }
    public  function apropos(){
        return view("pages.apropos");
    }
    public function services(){
        // $produits = DB::table('products')->inRandomOrder()->limit(1)->get();
        $produits = Product::orderBy('product_name', 'asc')->get();

        return view("pages.services")->with('produits', $produits);
    }
}
```

-Dans **services.blade.php**, commentez la ligne 15 :

```
PagesController.php   services.blade.php ×
resources > views > pages > services.blade.php > ...
 8
 9      <h1>Bienvenue sur la page Services</h1>
10      @foreach ($produits as $produit)
11          <div class="well">
12              <h1><a href="#">{{ $produit->product_name }}</a></h1>
13          </div>
14      @endforeach
15      {{-- {{ $produits->links() }} --}}
16  @endsection
17
```

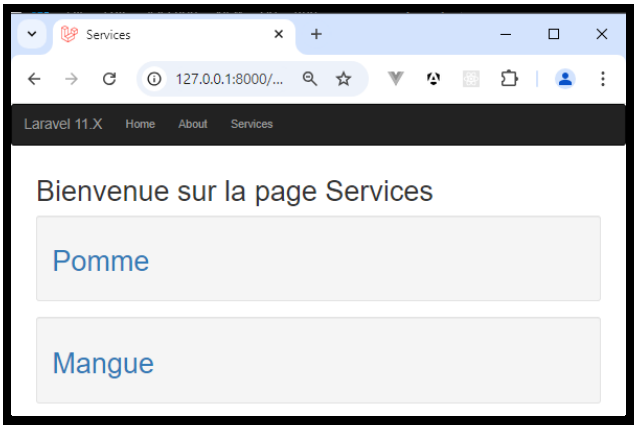
-Affichage dans le navigateur :



b- ordre décroissant

```
PagesController.php ×   services.blade.php ×
app > Http > Controllers > PagesController.php
11  {
13      public function  home() {
14          return view("pages.home");
15      }
16      public  function apropos(){
17          return view("pages.apropos");
18      }
19      public function services(){
20          // $produits = DB::table('products')->inRandomOrder()->limit(1)->get();
21          $produits = Product::orderBy('product_name', 'desc')->get();
22
23          return view("pages.services")->with('produits', $produits);
24      }
25  }
26
```

Résultat :



15. Affichage de données spécifiques (PARTIE 1)

Affichage du détail de chaque produit.

-Dans serices.blade.php :

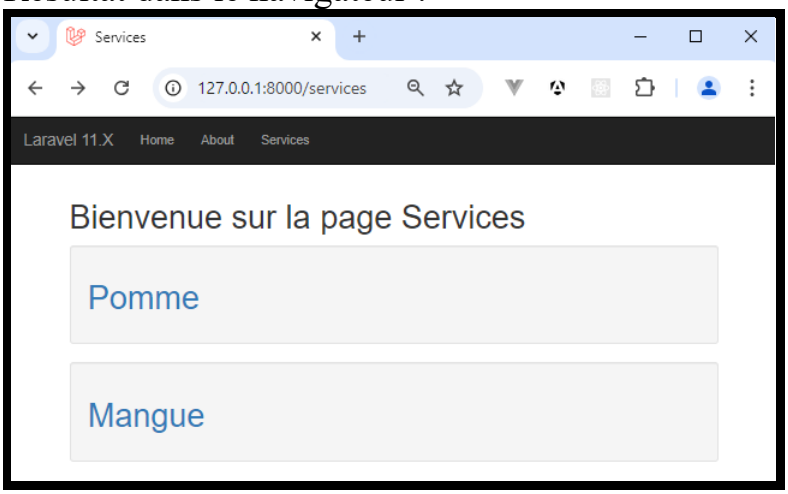
```blade
@extends('layouts.master')

@section('title')
    Services
@endsection

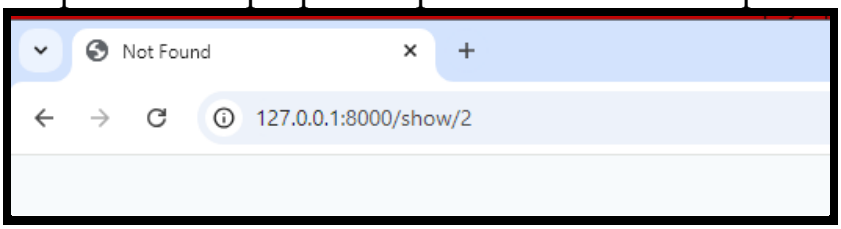
@section('contenu')

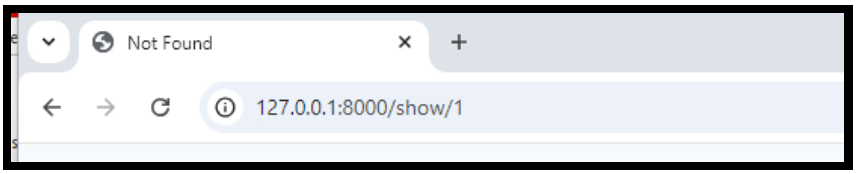
    <h1>Bienvenue sur la page Services</h1>
    @foreach ($produits as $produit)
        <div class="well">
            <h1><a href="/show/{{$produit->id }}">{{ $produit->product_name }}</a></h1>
        </div>
    @endforeach
    {{-- {{ $produits->links() }} --}}
@endsection
```

Résultat dans le navigateur :



Cliquez sur chaque produit pour afficher son url qui nous permet d'y accéder :





On voit que la route de chaque produit est affichée, donc on peut construire cette route dans **web.php** et en suite aller implémenter dans **PagesController** la méthode (show) correspondante :

-Dans **web.php**

```php
// Route::get('home/{id}/{nom}', function ($id, $nom) {
//     return "<h1><font color='red'>
//     Bonjour le monde !, mon id est : ".$id." et mon nom est :
// });

Route::get('/', [ PagesController::class, 'home']);

Route::get('/apropos', [ PagesController::class, 'apropos']);

Route::get('/services', [ PagesController::class, 'services']);

Route::get('/show/{id}', [ PagesController::class, 'show']);
```

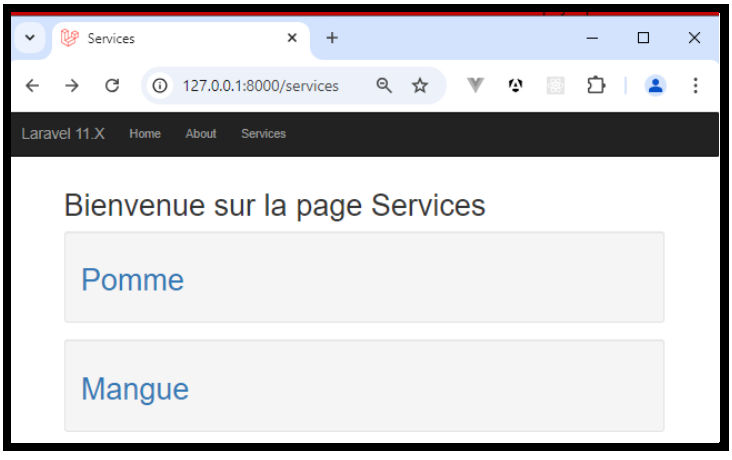
-Dans **PagesController.php**

Nous allons implémenter la méthode **show( )** et afficher l'id de chaque produit en cliquant sur ce dernier :

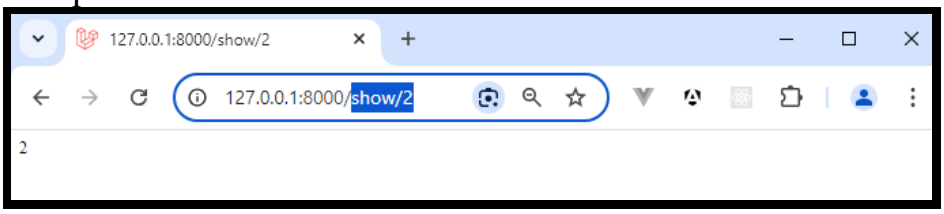
```php
{
    }
    public  function apropos(){
        return view("pages.apropos");
    }
    public function services(){
        // $produits = DB::table('products')->inRandomOrder()->limit(1)->get();
        $produits = Product::orderBy('product_name', 'desc')->get();

        return view("pages.services")->with('produits', $produits);
    }
    public function show($id){
        print($id);
    }
}
```

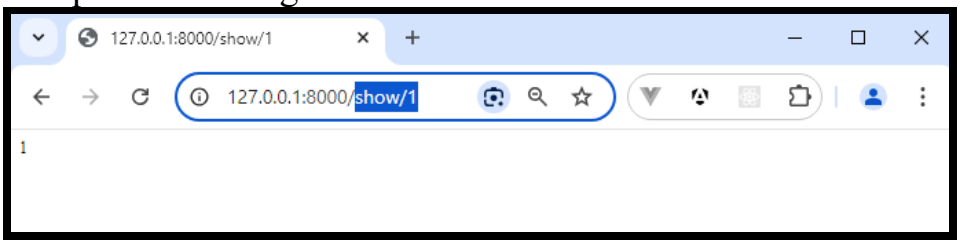
Résultat dans le navigateur :



-Cliquons sur Pomme :



-Cliquons sur mangue :



L'id du produit est bien affiché et c'est celui dans la base de données.

Donc on peut bien afficher toutes propriétés d'un produit.

NB :
N'oubliez pas que la page show n'est pas encore créée, il faut la créer dans **/views/pages/show.blade.php** et copier/coller le contenu de la page services.blade.php et la modifier :

16. Affichage de données spécifiques (PARTIE 2)

Suite :

show.blade.php

PagesController.php | show.blade.php | services.blade.php | web.php

resources > views > pages > show.blade.php > ...

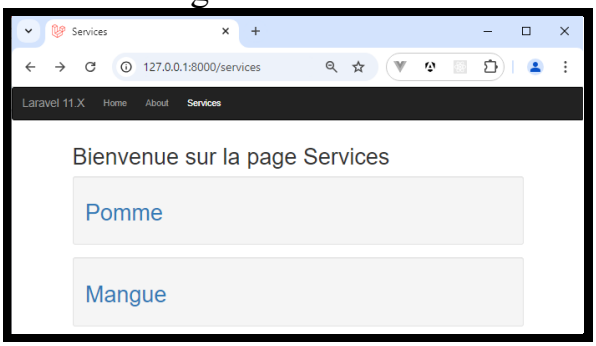
```
1  @extends('layouts.master')
2
3  @section('title')
4      Show
5  @endsection
6
7  @section('contenu')
8
9      <h1>Détail du produit</h1>
10     <hr>
11     <h1>{{ $produit->product_name}}</h1>
12     <h2>${{ $produit->product_price}}</h2>
13     <p>{{ $produit->product_description}}</p>
14     <hr>
15     <h4>{{ $produit->created_at }}</h4>
16     {{-- {{ $produits->links() }} --}}
17 @endsection
18
```

-Dans **PagesController** : implémentation de la méthode ***show( )***

```php
    }
    public  function apropos(){
        return view("pages.apropos");
    }
    public function services(){
        // $produits = DB::table('products')->inRandomOrder()->limit(1)->get();
        $produits = Product::orderBy('product_name', 'desc')->get();

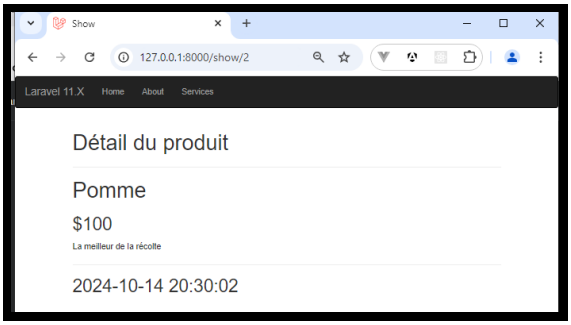
        return view("pages.services")->with('produits', $produits);
    }
    public function show($id){
        $produit = DB::table('products')
                    ->where('id', $id)
                    ->first();
        return view('pages.show')->with('produit', $produit);
    }
}
```

Dans le navigateur :

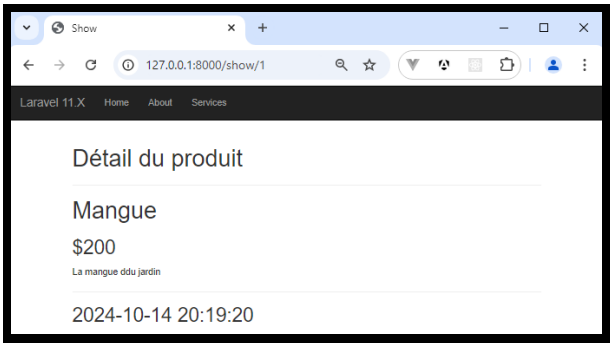


-Cliquer sur **Pomme** :

Résultat :



-Cliquez sur mangue :



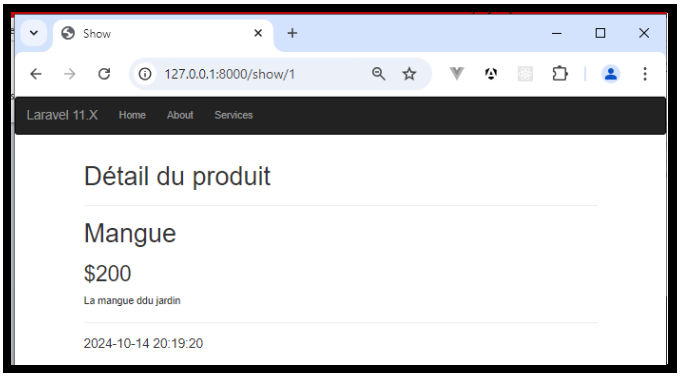
Utilisation de Laravel Eloquent :

-Dans **PagesController**

```php
{
    public function services(){
        $produits = Product::orderBy('product_name', 'desc')->get();

        return view("pages.services")->with('produits', $produits);
    }
    public  function show($id){
        // $produit = DB::table('products')
        //             ->where('id', $id)
        //             ->first();
        $produit = Product::find($id);
        return view('pages.show')->with('produit', $produit);
    }
}
```

Résultat :

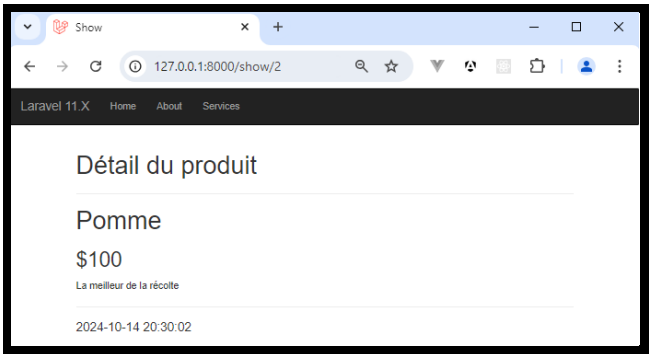


On obtient le même résultat et cette méthode est plus simple et a moins de code.

Autre manière de faire toujours dans **Eloquent :**

```php
{
    public function services(){
        return view("pages.services")->with('produits', $produits);
    }
    public  function show($id){
        // $produit = DB::table('products')
        //             ->where('id', $id)
        //             ->first();
        $produit = Product::where('id', $id)->first();
        return view('pages.show')->with('produit', $produit);
    }
}
```

Résultat :



Même résultat.

17. Sauvegarde de données avec le formulaire 1

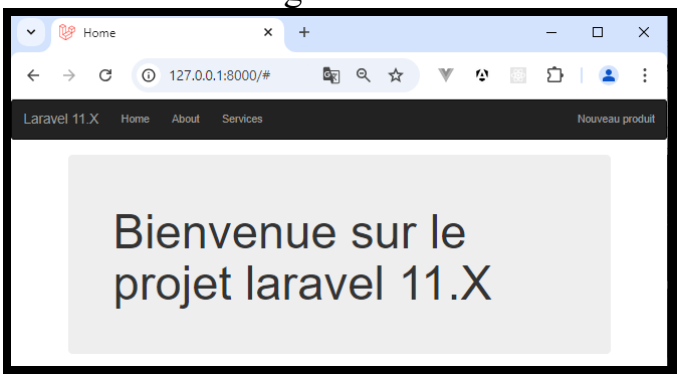
Comment créer les différents produits en utilisant des formulaires.

-Dans le fichier ***navbar.blade.php*,** il suffit de supprimer les commentaires sur la partie commentée.

```
<nav class="navbar navbar-inverse">
    <div class="container-fluid">
        <div class="navbar-header">
        <a class="navbar-brand" href="#">Laravel 11.X</a>
        </div>
        <ul class="nav navbar-nav">
            <li><a href="{{ URL::to("/") }}">Home</a></li>
            <li><a href="{{ url("/apropos") }}">About</a></li>
            <li><a href="{{ url("/services") }}">Services</a></li>
        </ul>
        <ul class="nav navbar-nav navbar-right">
            <li><a href="#">New product</a></li>
        </ul>
    </div>
</nav>
```

Changer *New product* en *Nouveau produit*.

-Actualisez le navigateur :

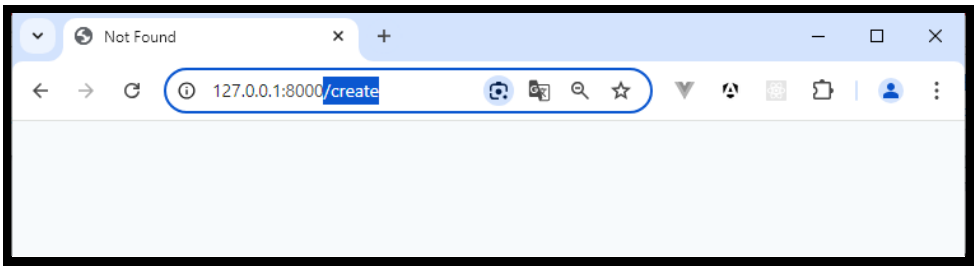


On voit la présence d'un nouveau menu en haut à droite : ***Nouveau produit***

-Dans **navbar.blade.php**, précisez la route :

```
<nav class="navbar navbar-inverse">
    <div class="container-fluid">
        <ul class="nav navbar-nav">
            <li><a href="{{ url("/apropos") }}">About</a></li>
            <li><a href="{{ url("/services") }}">Services</a></li>
        </ul>
        <ul class="nav navbar-nav navbar-right">
            <li><a href="{{ url('/create')}}">Nouveau produit</a></li>
        </ul>
    </div>
</nav>
```

-Dans le navigateur, cliquez sur le menu **Nouveau produit**



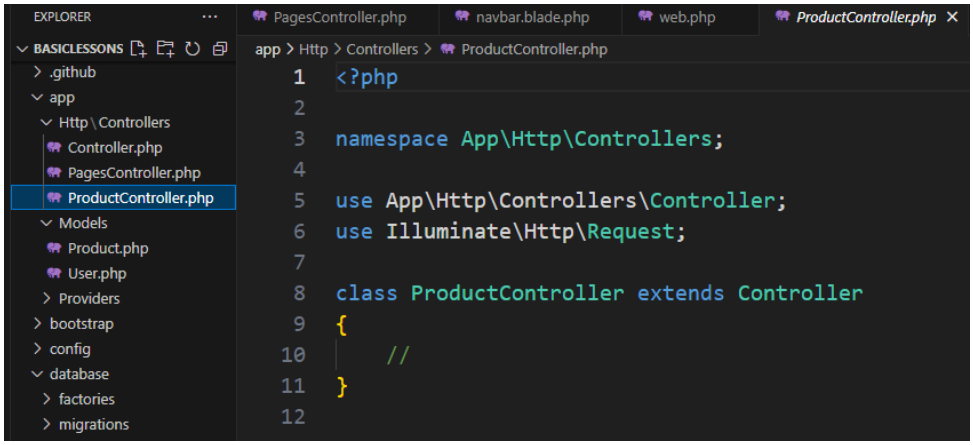
On récupère la route affichée pour aller l'implémenter dans **web.blade.php** :
Avant tout cela, créons en ligne de commande un autre contrôleur que nous allons appeler : **ProductController**

```
C:\xampp\htdocs\basiclessons>php artisan make:controller ProductController

   INFO  Controller [C:\xampp\htdocs\basiclessons\app\Http\Controllers\ProductController.php] created successfully.

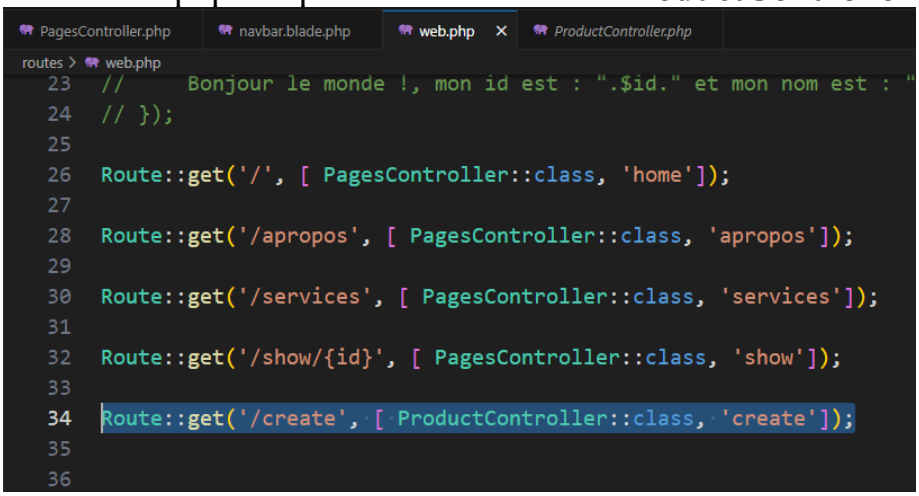

C:\xampp\htdocs\basiclessons>
```

Vérification :



Maintenant on peut utiliser **ProductController** dans la route :

-Dans web.php : importer dans l'entête : **ProductController**



-Dans le dossier pages, créer le fichier ***create.blade.php***

Copier/coller le contenu du fichier form.html donné par le formateur dans la partie contenu du fichier **create.blade.php** :

PagesController.php | ProductController.php 1 | create.blade.php

resources > views > pages > create.blade.php > form.form-horizontal > div.form-group

```
@extends('layouts.master')

@section('title')
   Create
@endsection

@section('contenu')

<form action="{{url('/saveproduct')}}"
            method="POST" class="form-horizontal">
    {{-- {{csrf_field()}} --}}
    @csrf
    <div class="form-group">
        <label>Product</label>
        <input type="text" name="product_name"
            placeholder="Product Name" class="form-control">
    </div>
    <div class="form-group">
        <label>Product Price</label>
        <input type="text" name="product_price"
            placeholder="Product Price" class="form-control">
    </div>
  <div class="form-group">
     <label>Product description</label>
     <textarea name="product_description" cols="30" rows="10"></textarea>
  </div>
  <input type="submit" value="Add Product" class="btn btn-primary">
</form>

@endsection

```

-Dans **ProductController** (nouvellement créé), implémenter la méthode **create**

PagesController.php | ProductController.php | create.blade.php

app > Http > Controllers > ProductController.php

```
<?php

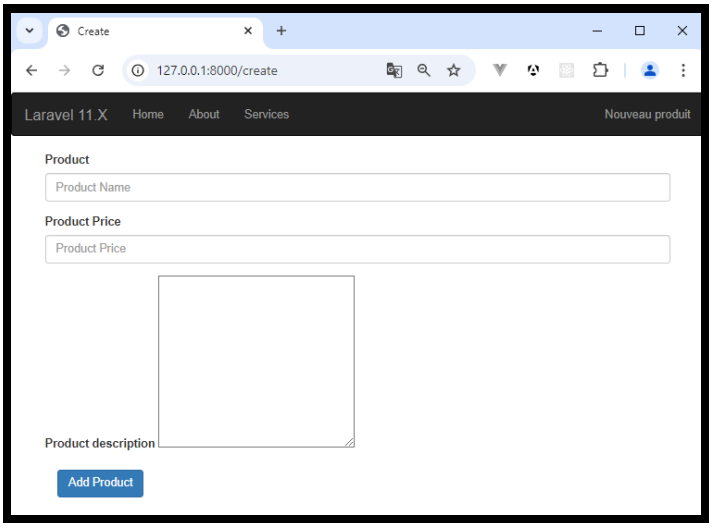
namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;

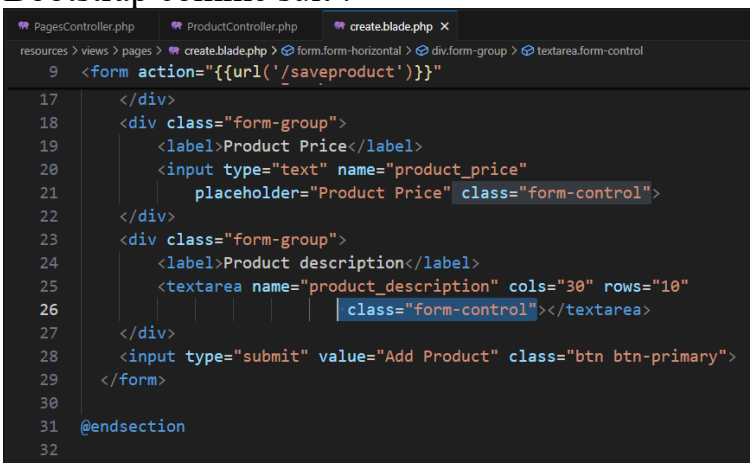
class ProductController extends Controller
{
    //
    public function create(){
        return view('pages.create');
    }
}

```

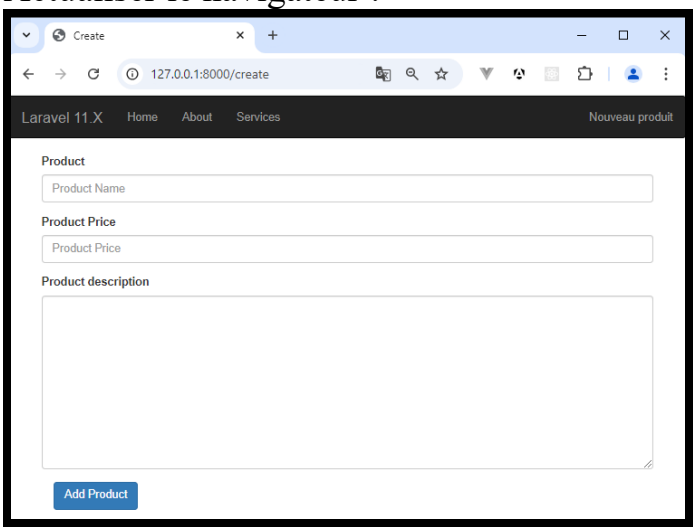
-Dans le navigateur :



On a un petit problème au niveau du textArea, il faut ajouter juste une classe Bootstrap comme suit :

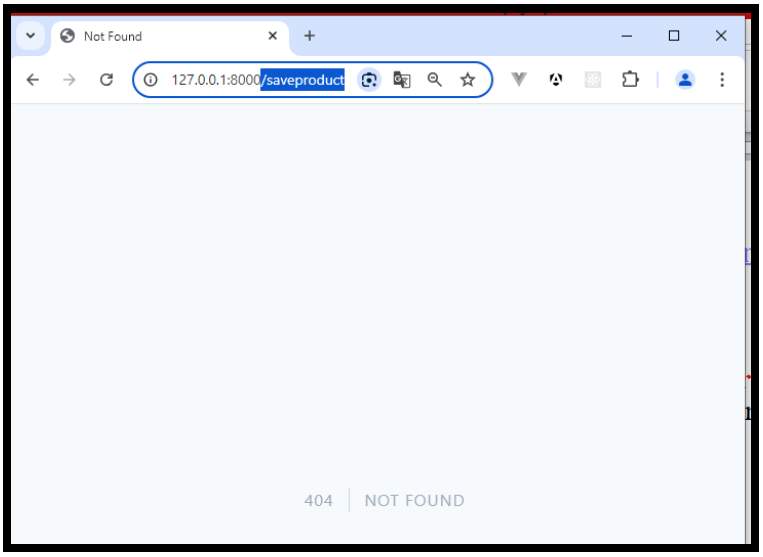


Actualiser le navigateur :



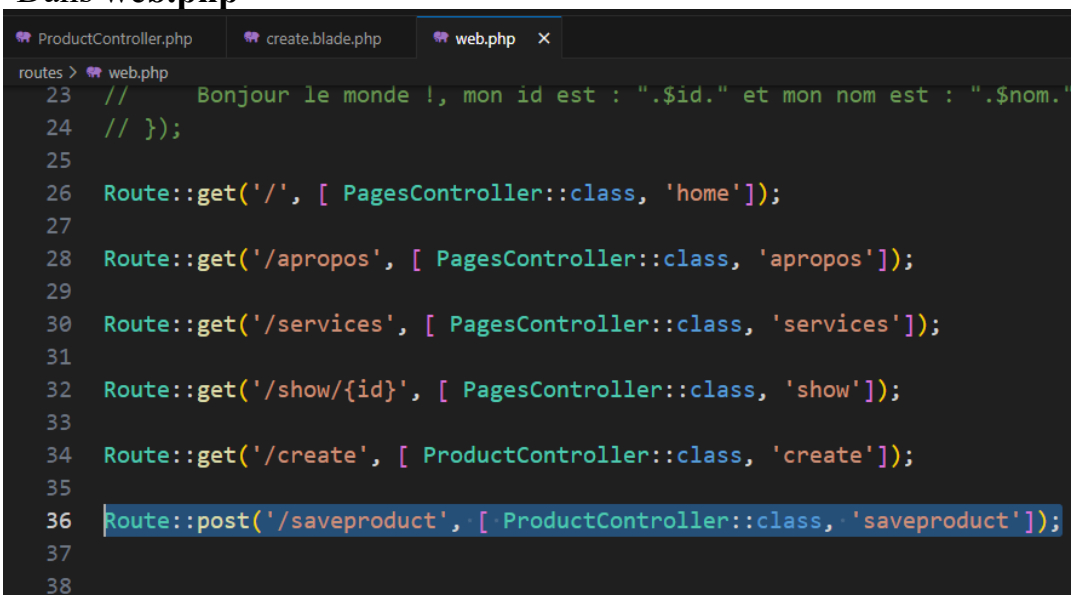
18. Sauvegarde de données avec le formulaire 2

On remarque que lorsqu'on clique sur le bouton **Add product** on obtient la page suivante :



Donc il faut aller implémenter la route affichée dans l'url. D'ailleurs quand on regarde dans le code du formulaire, on voit la route dans l'action du formulaire.

-Dans **web.php**



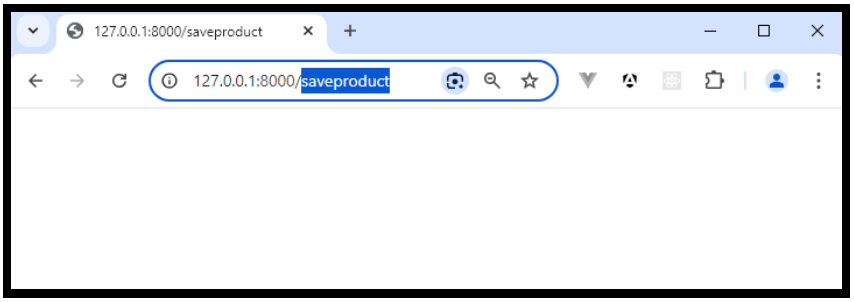
-Dans **ProductController** :
Il va récupérer toutes les valeurs de la propriété ***name*** dans le formulaire, les stocker dans l'objet **$request** qui est en argument de la fonction ***saveproduct( )*** du contrôleur **PagesController**. C'est à lui de les injecter dans la base de données.
Affichons le contenu de **product_name** à l'aide de ***print( )*** :

```php
use App\Http\Controllers\Controller;
use Illuminate\Http\Request;

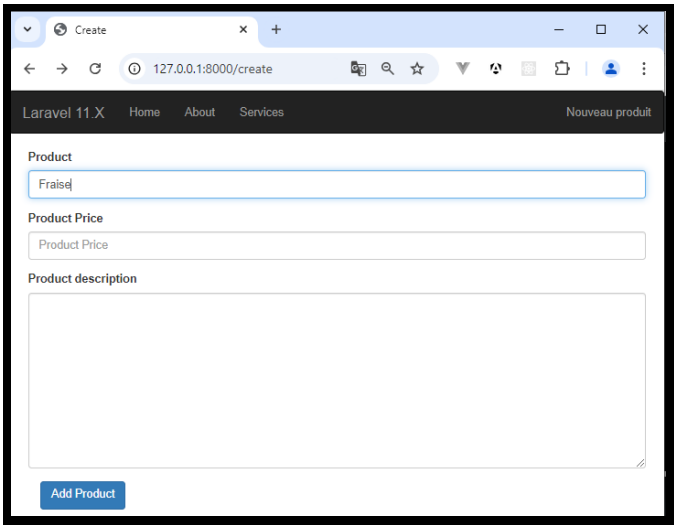
class ProductController extends Controller
{
    //
    public function create(){
        return view('pages.create');
    }

    public function saveproduct(request $request){
        print($request->product_name);
    }
}
```

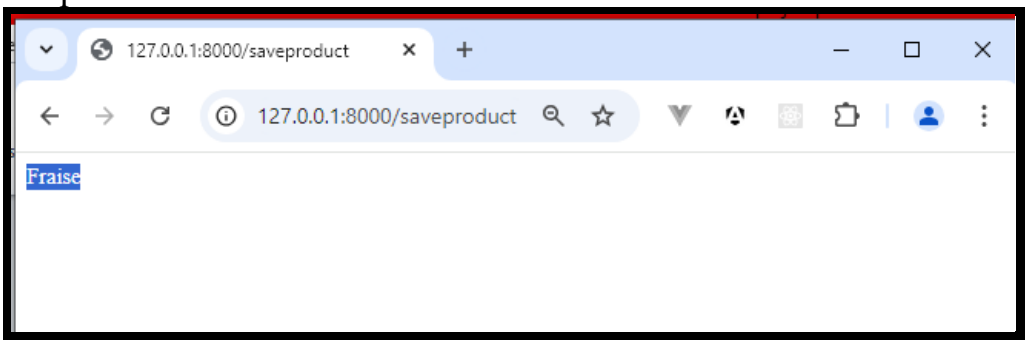
Revenons sur le navigateur et cliquons sur **Add Product** sans rien saisir dans le formulaire :



-Cette fois remplissons le formulaire et cliquons sur **Add Product** :

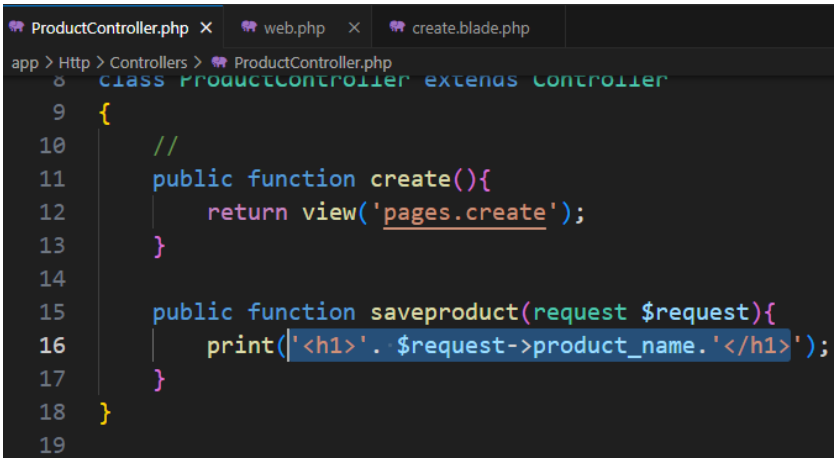


Cliquons sur **Add Product**

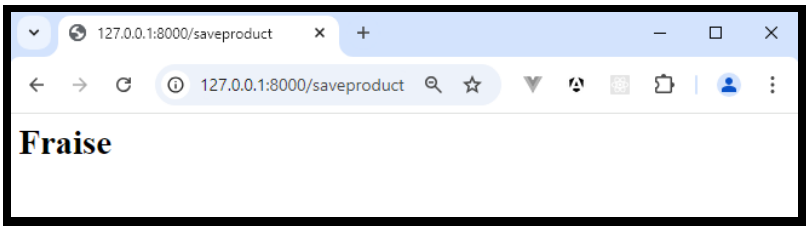


Le formulaire affiche la saisie.

Pour mieux voir :



Affichons :



Voici l'importance de cet objet **$request** dans le code.

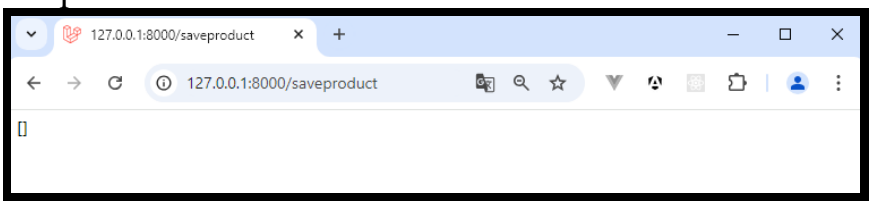
Nous allons insérer ces valeurs dans la base de données.
Revenant dans **PagesController**.
On utilise la méthode Eloquent :

```php
class ProductController extends Controller
{
    //
    public function create(){
        return view('pages.create');
    }

    public function saveproduct(request $request){
        $product = new Product();
        print($product);

    }
}
```

Cliquons et voir le résultat affiché



On obtient un tableau vide.

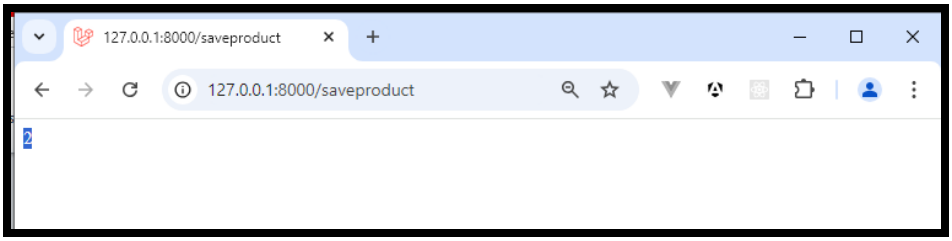
Maintenant, utilisons la méthode ***count( )*** sur l'objet :

```php
class ProductController extends Controller
{
    //
    public function create(){
        return view('pages.create');
    }

    public function saveproduct(request $request){
        $product = new Product();
        print($product::count());

    }
}
```

Résultat :



Il renvoie 2, correspondant effectivement au nombre de produits existant dans la table **$product**.

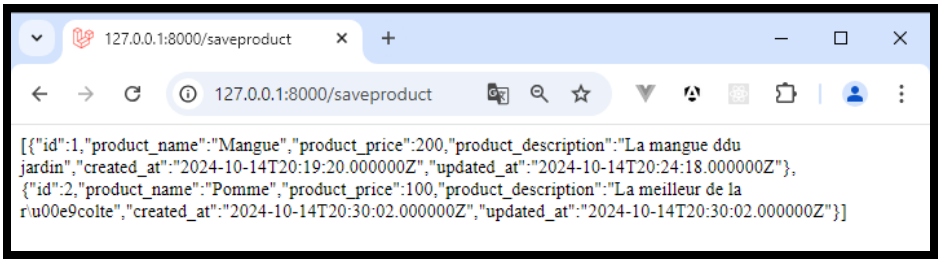
Essayons la méthode get() sur $product :

```php
class ProductController extends Controller
{
    //
    public function create(){
        return view('pages.create');
    }

    public function saveproduct(request $request){
        $product = new Product();
        print($product::get());

    }
}
```

Résultat :



Mais nous voulons le contenu de notre table dans la base de données :

19. Sauvegarde de données avec le formulaire 3

Suite :

```php
use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use App\Models\Product;

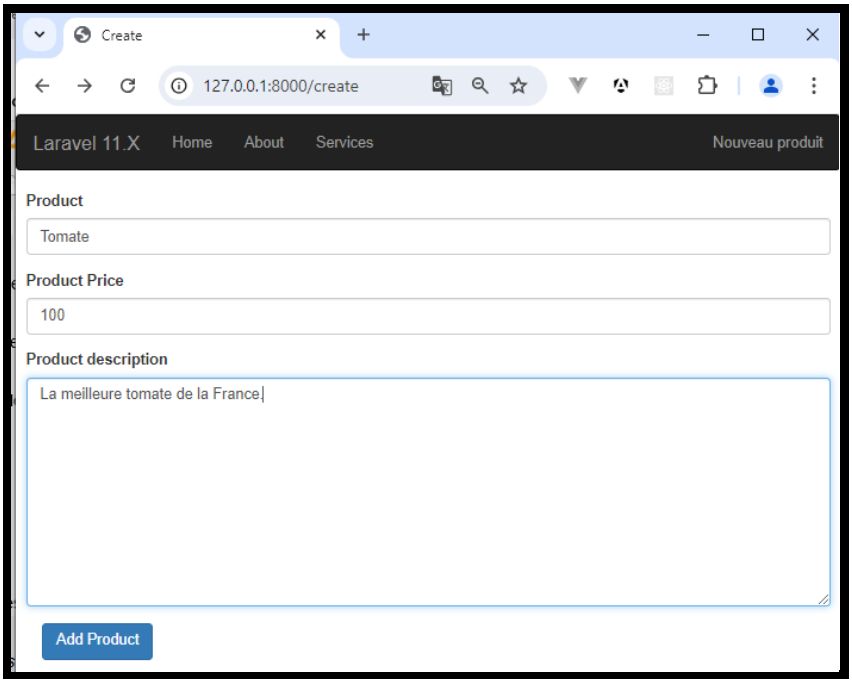
class ProductController extends Controller
{
    //
    public function create(){
        return view('pages.create');
    }

    public function saveproduct(request $request){
        $product = new Product();

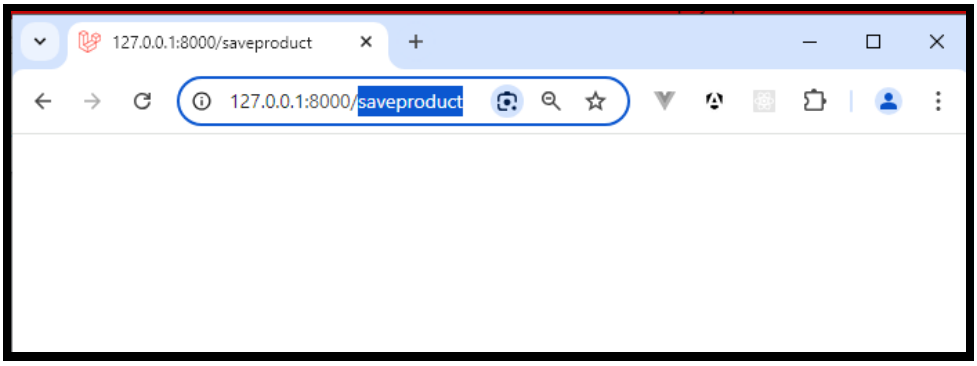
        $product->product_name = $request->product_name;
        $product->product_price = $request->product_price;
        $product->product_description = $request->product_description;

        $product->save();
    }
}
```

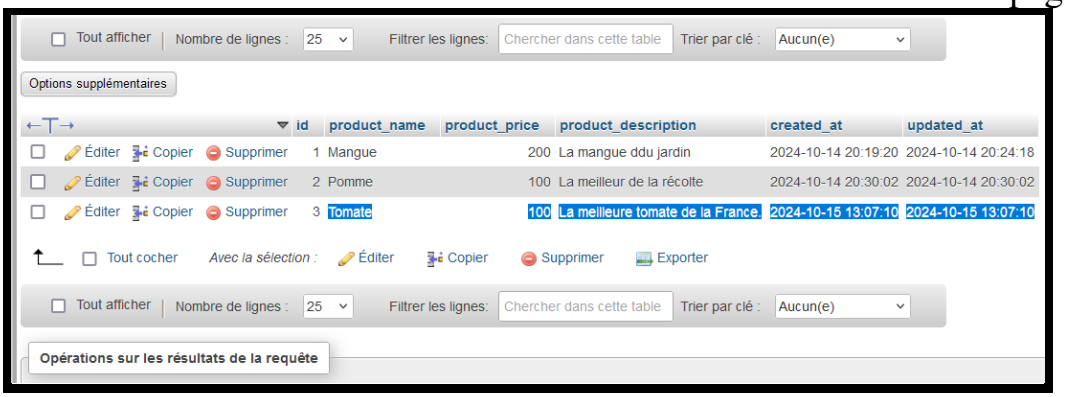
Testons en insérant un produit :



Cliquons sur **Add Product** :
Résultat :



On obtient une page blanche.
Allons vérifier au niveau de la base de données en rafraichissant la page :



On obtient bien un troisième enregistrement dans la base de données.
Après la sauvegarde, c'est une page vierge qui est retournée, on pouvait faire une redirection.
Nous allons faire une redirection sur le formulaire (d'où est ce que tu viens) :

```php
{
    public function create(){
    }

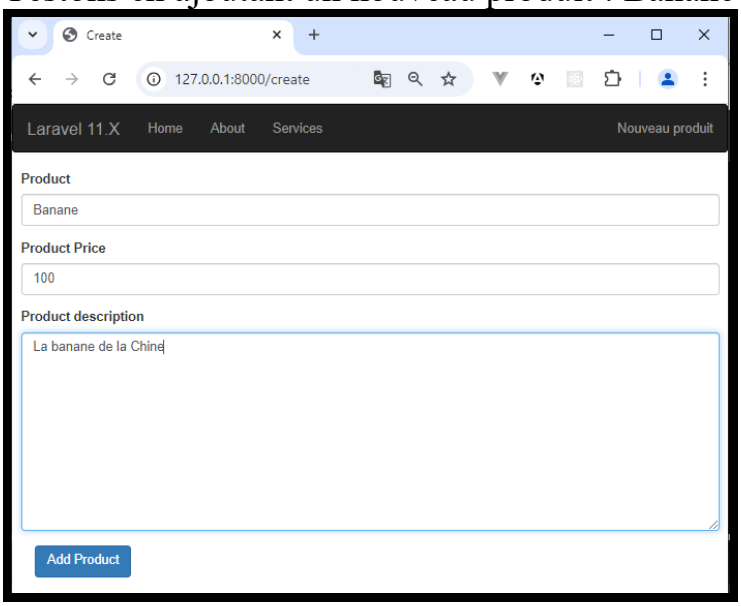
    public function saveproduct(request $request){
        $product = new Product();

        $product->product_name = $request->product_name;
        $product->product_price = $request->product_price;
        $product->product_description = $request->product_description;

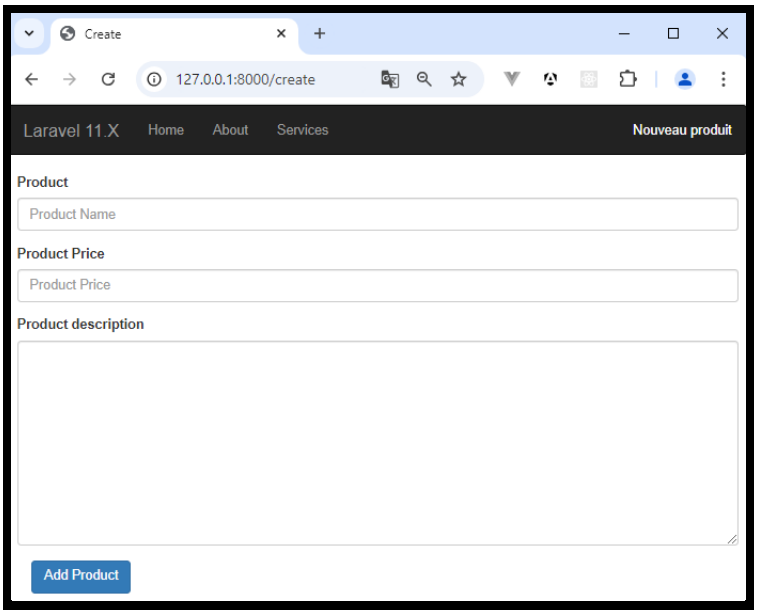
        $product->save();

        return back();
    }
}
```

Testons en ajoutant un nouveau produit : Banane



Cliquez sur **AddProdact**
Résultat : on revient sur le formulaire



On vérifie au niveau de la base de données :



	id	product_name	product_price	product_description	created_at	updated_at
Éditer Copier Supprimer	1	Mangue	200	La mangue ddu jardin	2024-10-14 20:19:20	2024-10-14 20:24:18
Éditer Copier Supprimer	2	Pomme	100	La meilleur de la récolte	2024-10-14 20:30:02	2024-10-14 20:30:02
Éditer Copier Supprimer	3	Tomate	100	La meilleure tomate de la France.	2024-10-15 13:07:10	2024-10-15 13:07:10
Éditer Copier Supprimer	4	Banane	100	La banane de la Chine	2024-10-15 17:24:56	2024-10-15 17:24:56

Tout cocher Avec la sélection : Éditer Copier Supprimer Exporter

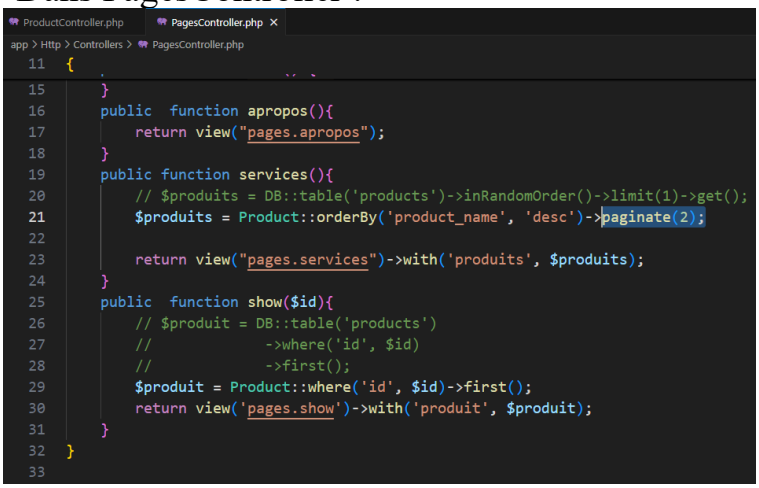
Tout afficher | Nombre de lignes : 25 Filtrer les lignes: Chercher dans cette table Trier par clé : Aucun(e)

Opérations sur les résultats de la requête

Un 4ieme enregistrement dans la base. On peut le vérifier aussi sur la page services.

Affichons par 2 éléments par page, pour cela :

-Dans PagesController :



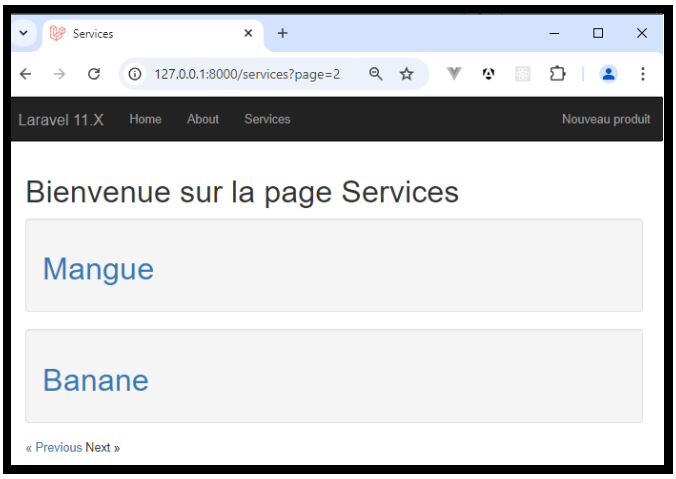
```
ProductController.php  PagesController.php ×
app > Http > Controllers > PagesController.php
11  {
15      }
16      public  function apropos(){
17          return view("pages.apropos");
18      }
19      public function services(){
20          // $produits = DB::table('products')->inRandomOrder()->limit(1)->get();
21          $produits = Product::orderBy('product_name', 'desc')->paginate(2);
22
23          return view("pages.services")->with('produits', $produits);
24      }
25      public  function show($id){
26          // $produit = DB::table('products')
27          //             ->where('id', $id)
28          //             ->first();
29          $produit = Product::where('id', $id)->first();
30          return view('pages.show')->with('produit', $produit);
31      }
32  }
33
```

-Dans services : supprimer le commentaire (ligne 15)



```
ProductController.php  PagesController.php  services.blade.php ×
resources > views > pages > services.blade.php > ...
1   @extends('layouts.master')
2
3   @section('title')
4       Services
5   @endsection
6
7   @section('contenu')
8       <h1>Bienvenue sur la page Services</h1>
9       @foreach ($produits as $produit)
10          <div class="well">
11              <h1><a href="/show/{{$produit->id }}">
12                  {{ $produit->product_name }}</a></h1>
13          </div>
14      @endforeach
15      {{ $produits->links() }}
16  @endsection
17
```

-Actualiser le navigateur :



Tout fonctionne correctement.

Si après insertion on veut se rediriger vers la page **services** :

```
ProductController.php | PagesController.php | services.blade.php
app > Http > Controllers > ProductController.php
10  {
16      public function saveproduct(request $request){
17          $product = new Product();
18
19          $product->product_name = $request->product_name;
20          $product->product_price = $request->product_price;
21          $product->product_description = $request->product_description;
22
23          $product->save();
24
25          return redirect('/services');
26      }
27  }
28
```

-Une manière de montrer que la requête vient d'un input (tel est notre cas de formulaire) :

```
ProductController.php | PagesController.php | services.blade.php
app > Http > Controllers > ProductController.php
10  {
15
16      public function saveproduct(request $request){
17          $product = new Product();
18
19          $product->product_name = $request->input('product_name');
20          $product->product_price = $request->input('product_price');
21          $product->product_description = $request->input('product_description');
22
23          $product->save();
24
25          return redirect('/services');
26      }
27  }
28
```

Maintenant de retour sur la page services, on aimerait afficher un message : *tel produit a été ajouté avec succès.*

-Dans **ProductController** :

```php
{
    public function saveproduct(request $request){
        $product->product_name = $request->input('product_name');
        $product->product_price = $request->input('product_price');
        $product->product_description = $request->input('product_description');

        $product->save();

        return redirect('/services')->with('status', 'Le produit'.
        $request->input('product_name').' a été inséré avec succès !!!');
    }
}
```

-Dans **services.blade.php**

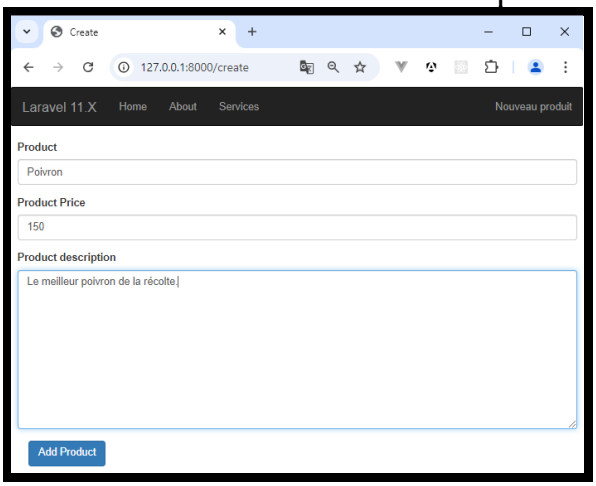
```blade
@endsection

@section('contenu')
    <h1>Bienvenue sur la page Services</h1>

    @if (Session::has('status'))
        <div class="alert alert-success">
            {{ Session::get('status')}}
        </div>
    @endif

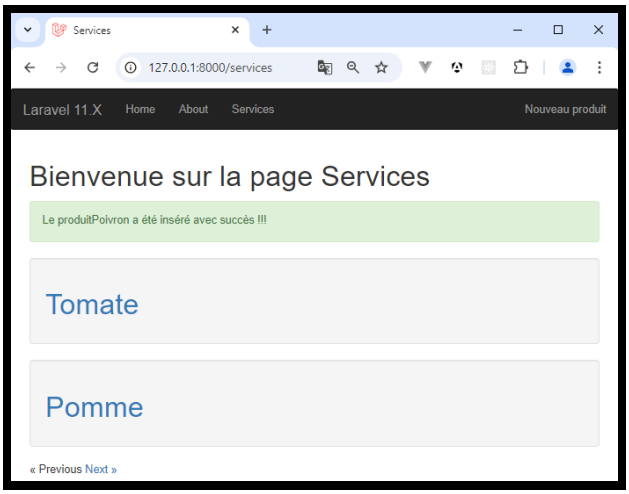
    @foreach ($produits as $produit)
        <div class="well">
            <h1><a href="/show/{{$produit->id }}">
                {{ $produit->product_name }}</a></h1>
        </div>
    @endforeach
    {{ $produits->links() }}
@endsection
```

-Testons en insérant un nouveau produit : Poivron

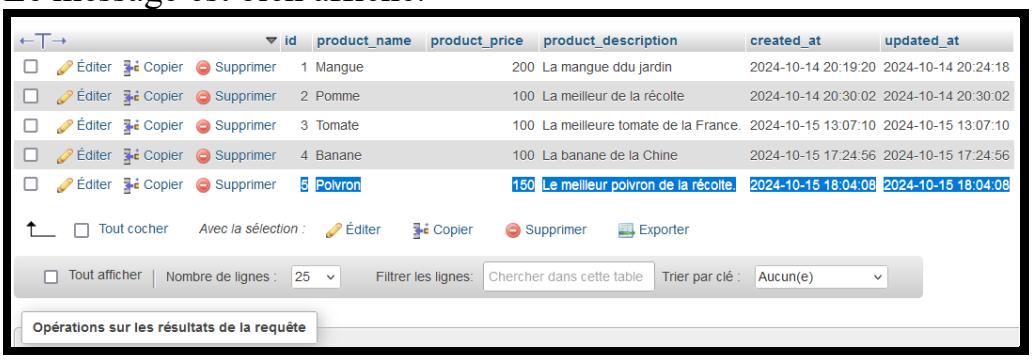


Cliquons sur **Add Product** :

Résultat :



Le message est bien affiché.



	id	product_name	product_price	product_description	created_at	updated_at
Éditer Copier Supprimer	1	Mangue	200	La mangue ddu jardin	2024-10-14 20:19:20	2024-10-14 20:24:18
Éditer Copier Supprimer	2	Pomme	100	La meilleur de la récolte	2024-10-14 20:30:02	2024-10-14 20:30:02
Éditer Copier Supprimer	3	Tomate	100	La meilleure tomate de la France.	2024-10-15 13:07:10	2024-10-15 13:07:10
Éditer Copier Supprimer	4	Banane	100	La banane de la Chine	2024-10-15 17:24:56	2024-10-15 17:24:56
Éditer Copier Supprimer	5	Poivron	150	Le meilleur poivron de la récolte.	2024-10-15 18:04:08	2024-10-15 18:04:08

Tout cocher *Avec la sélection :* Éditer Copier Supprimer Exporter

Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table | Trier par clé : Aucun(e)

Opérations sur les résultats de la requête

20. Sauvegarde de données avec le formulaire 4

QueryBuilder : interaction entre le contrôleur et la table Product dans la BDD
Pour cela il faut établir la connexion entre le contrôleur et la BDD en important dans l'entête du contrôleur la DB.

ProductController.php

ProductController.php × services.blade.php

app > Http > Controllers > ProductController.php

```php
<?php
namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;
use App\Models\Product;

class ProductController extends Controller
{
    //
    public function create(){
        return view('pages.create');
    }
    public function saveproduct(request $request){
        // $product = new Product();
        // $product->product_name = $request->input('product_name');
        // $product->product_price = $request->input('product_price');
        // $product->product_description = $request->input('product_description');

        // $product->save();

```

```php
        // Début : Query Builder
        //Déclarer une variable qui va contenir toutes
        //les valeurs des input du formulaire

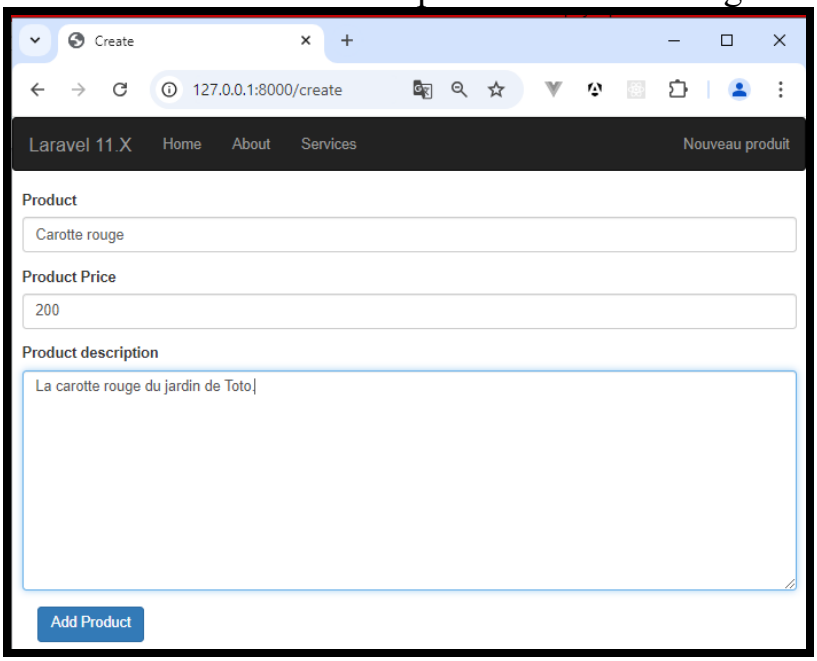
        $data = array();
        $data['product_name'] = $request->input('product_name');
        $data['product_price'] = $request->input('product_price');
        $data['product_description'] = $request->input('product_description');

        DB::table('products')->insert($data);

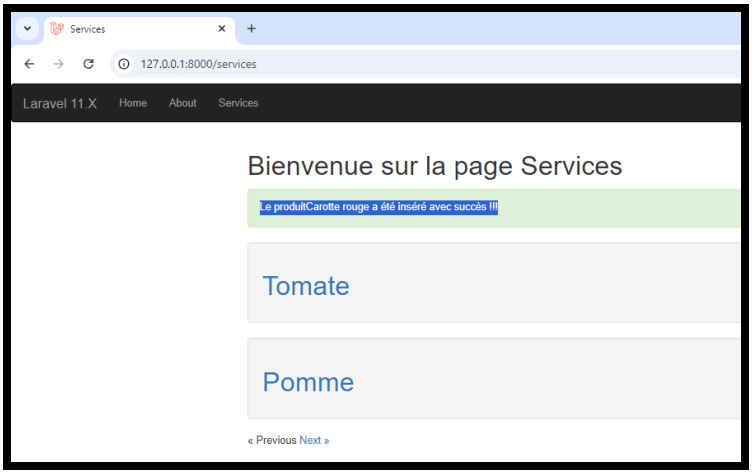
        return redirect('/services')->with('status', 'Le produit'.
        $request->input('product_name').' a été inséré avec succès !!!');
    }
}

```

Test : insérons un nouveau produit : Carotte rouge

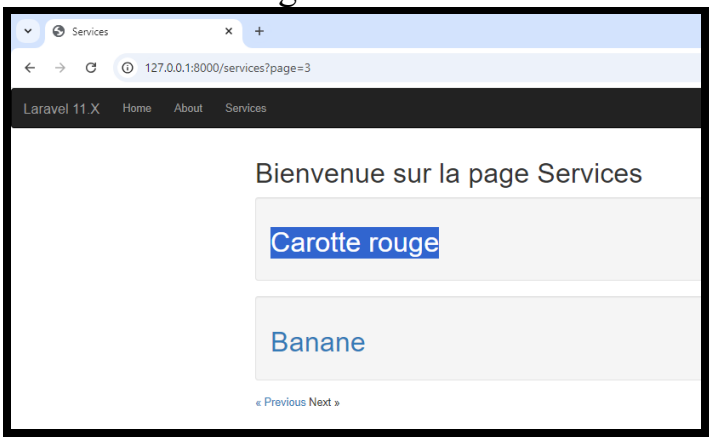


Cliquez sur **Add Product**
Résultat :



Vérification :

Au niveau du navigateur :



Au niveau de la base de données :



	id	product_name	product_price	product_description	created_at	updated_at
Éditer Copier Supprimer	1	Mangue	200	La mangue ddu jardin	2024-10-14 20:19:20	2024-10-14 20:24:18
Éditer Copier Supprimer	2	Pomme	100	La meilleur de la récolte	2024-10-14 20:30:02	2024-10-14 20:30:02
Éditer Copier Supprimer	3	Tomate	100	La meilleure tomate de la France.	2024-10-15 13:07:10	2024-10-15 13:07:10
Éditer Copier Supprimer	4	Banane	100	La banane de la Chine	2024-10-15 17:24:56	2024-10-15 17:24:56
Éditer Copier Supprimer	5	Poivron	150	Le meilleur poivron de la récolte.	2024-10-15 18:04:08	2024-10-15 18:04:08
Éditer Copier Supprimer	7	Carotte rouge	200	La carotte rouge du jardin de Toto.	NULL	NULL

Tout cocher Avec la sélection : Éditer Copier Supprimer Exporter

On remarque que les timestemps ne sont pas affichés.

Validation :

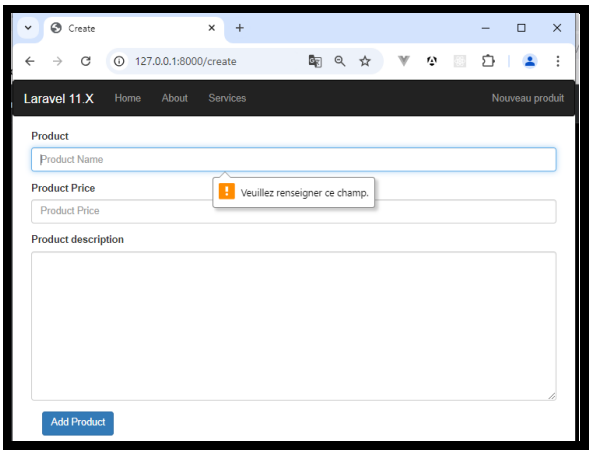
-Dans **create.blade.php**

ProductController.php | services.blade.php | create.blade.php

resources > views > pages > create.blade.php > ...

```
 9  <form action="{{url('/saveproduct')}}"
13      <div class="form-group">
14          <label>Product</label>
15          <input type="text" name="product_name"
16              placeholder="Product Name" class="form-control" required>
17      </div>
18      <div class="form-group">
19          <label>Product Price</label>
20          <input type="text" name="product_price"
21              placeholder="Product Price" class="form-control" required>
22      </div>
23      <div class="form-group">
24          <label>Product description</label>
25          <textarea name="product_description" cols="30" rows="10"
26                              class="form-control" required></textarea>
27      </div>
28      <input type="submit" value="Add Product" class="btn btn-primary">
29    </form>
30
31  @endsection
32
```

Pour tester la validation, on affiche le formulaire et on essaye d'envoyer sans saisir :



Cette manière de validation (côté client) peut être by-passée. Donc il faut la faire dans le Controller (côté serveur).

Nous allons faire appel à la fonction valida*te( )* :

```php
<?php
namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;
use App\Models\Product;

class ProductController extends Controller
{
    //
    public function create(){
        return view('pages.create');
    }
    public function saveproduct(Request $request){

        $request->validate([
            'product_name' => 'required',
            'product_price' => 'required',
            'product_description' => 'required',
        ]);

    // $product = new Product();
    // $product->product_name = $request->input('product_name');
    // $product->product_price = $request->input('product_price');
    // $product->product_description = $request->input('product_description');

    // $product->save();

    // Début : Query Builder
    //Déclarer une variable qui va contenir toutes
    //les valeurs des input du formulaire

    $data = array();
    $data['product_name'] = $request->input('product_name');
    $data['product_price'] = $request->input('product_price');
    $data['product_description'] = $request->input('product_description');

    DB::table('products')->insert($data);

    return redirect('/services')->with('status', 'Le produit'.
    $request->input('product_name').' a été inséré avec succès !!!');
    }
}
```

Maintenant, pour tester, il faut :

- supprimer ***required*** placé dans **create.php.blade.php**
- Commenter dans **ProductController** comme ci-dessous :

ProductController.php | services.blade.php | create.blade.php

app > Http > Controllers > ProductController.php

```php
<?php
namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;
use App\Models\Product;

class ProductController extends Controller
{
    //
    public function create(){
        return view('pages.create');
    }
    public function saveproduct(Request $request){

        // $request->validate([
        //     'product_name' => 'required',
        //     'product_price' => 'required',
        //     'product_description' => 'required',
        // ]);

    // $product = new Product();
    // $product->product_name = $request->input('product_name');
    // $product->product_price = $request->input('product_price');
    // $product->product_description = $request->input('product_description');

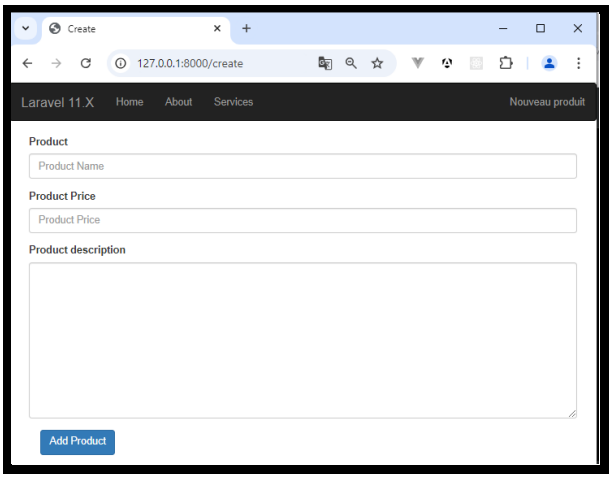
    // $product->save();

    // Début : Query Builder
    //Déclarer une variable qui va contenir toutes
    //les valeurs des input du formulaire

//     $data = array();
//     $data['product_name'] = $request->input('product_name');
//     $data['product_price'] = $request->input('product_price');
//     $data['product_description'] = $request->input('product_description');

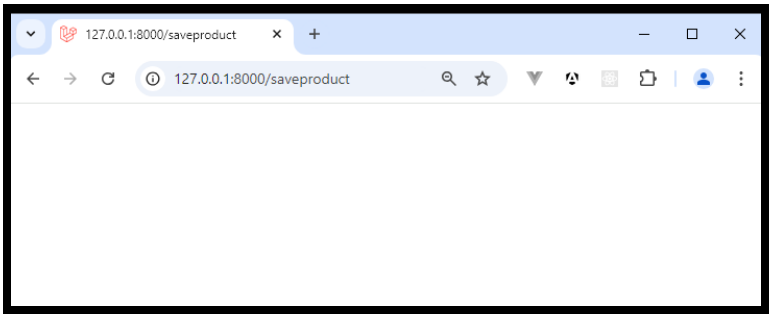
//     DB::table('products')->insert($data);

//     return redirect('/services')->with('status', 'Le produit '.
//     $request->input('product_name').' a été inséré avec succès !!!');
    }
}
```



Cliquons sur Add Product :

Résultat :



On obtient une page vierge.

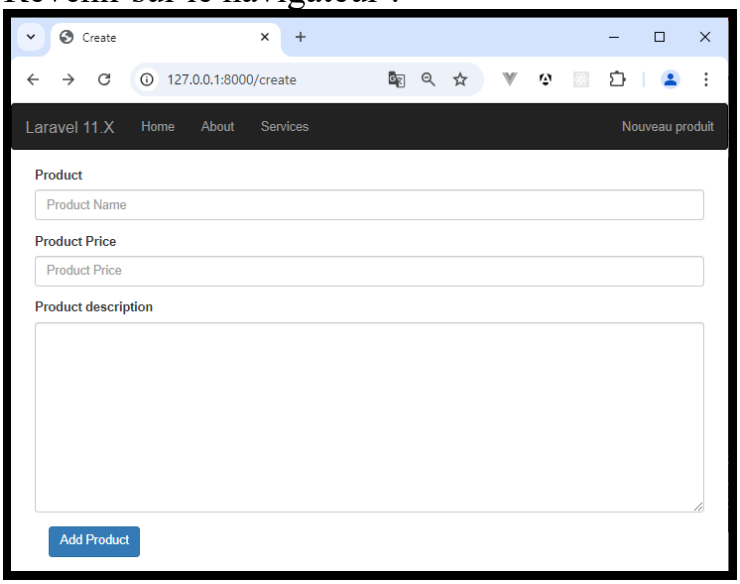
-Décommentons dans ProductController comme suit :

```php
class ProductController extends Controller
{
    //
    public function create(){
        return view('pages.create');
    }
    public function saveproduct(request $request){

        $this->validate($request, [
            'product_name' => 'required',
            'product_price' => 'required',
            'product_description' => 'required',
        ]);

        // $product = new Product();
        // $product->product_name = $request->input('produ
        // $product->product_price = $request->input('prod
        // $product->product_description = $request->input
```

Revenir sur le navigateur :



Cliquons sur Add Product :

Résultat :

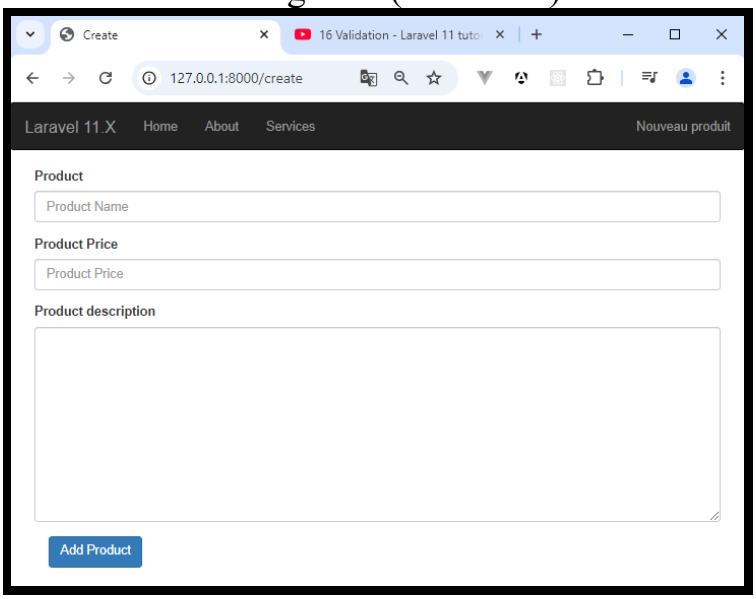
On revient sur la même page (rien ne se passe) pour me dire qu'il y a une erreur.

Comment afficher cette erreur ?

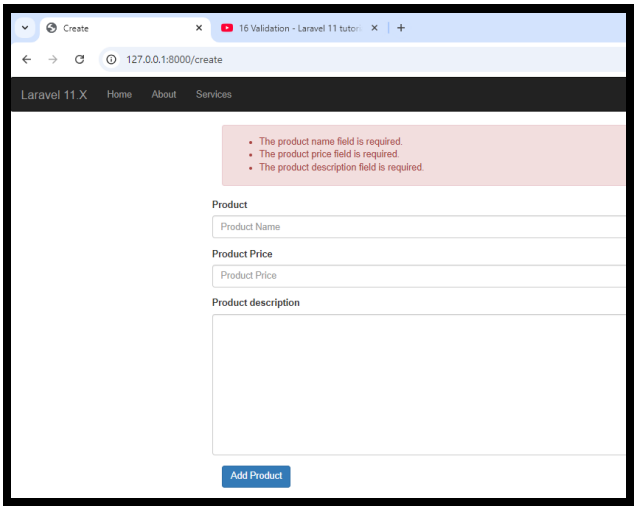
-Dans **create.blade.php** :

```
ProductController.php | services.blade.php | create.blade.php
resources > views > pages > create.blade.php > ...
1  @extends('layouts.master')
2
3  @section('title')
4      Create
5  @endsection
6
7  @section('contenu')
8
9  @if (count($errors) > 0)
10     <div class="alert alert-danger">
11         <ul>
12             @foreach ($errors->all() as $error)
13             <li>{{ $error }}</li>
14             @endforeach
15         </ul>
16     </div>
17 @endif
18
19 <form action="{{url('/saveproduct')}}"
20                 method="POST" class="form-horizontal">
21     {{-- {{csrf_field()}} --}}
22     @csrf
23     <div class="form-group">
24         <label>Product</label>
```

Revenir sur le navigateur (actualisez)

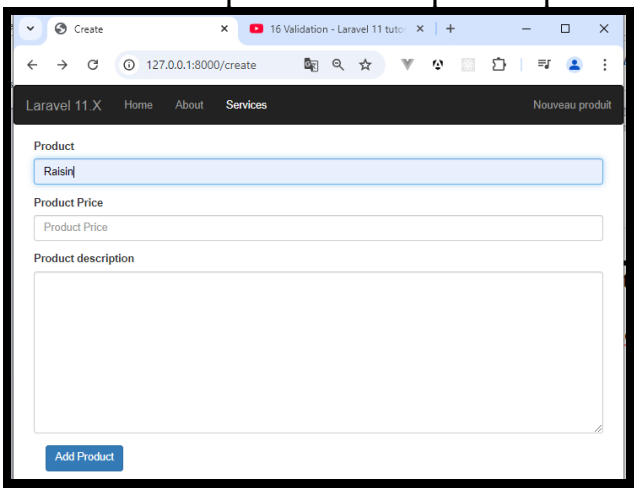


Cliquez sur Add Product
Résultat :

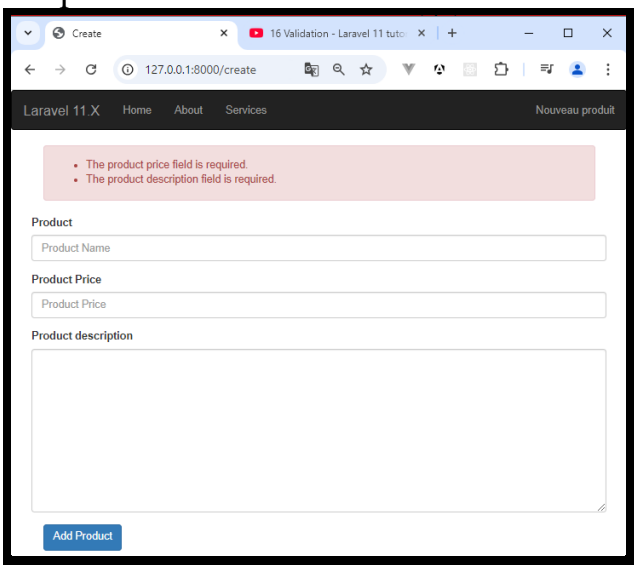


Les messages d'erreurs s'affichent correctement.

-Saisir dans le premier champ et cliquez sur Add Product :

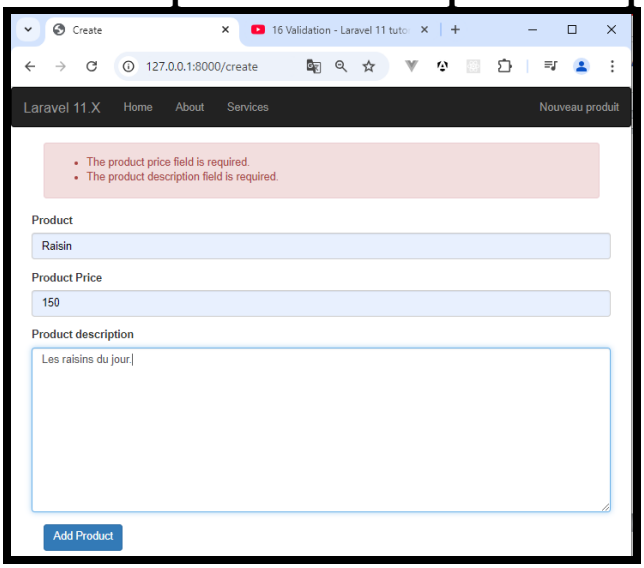


Cliquez sur Add Product :

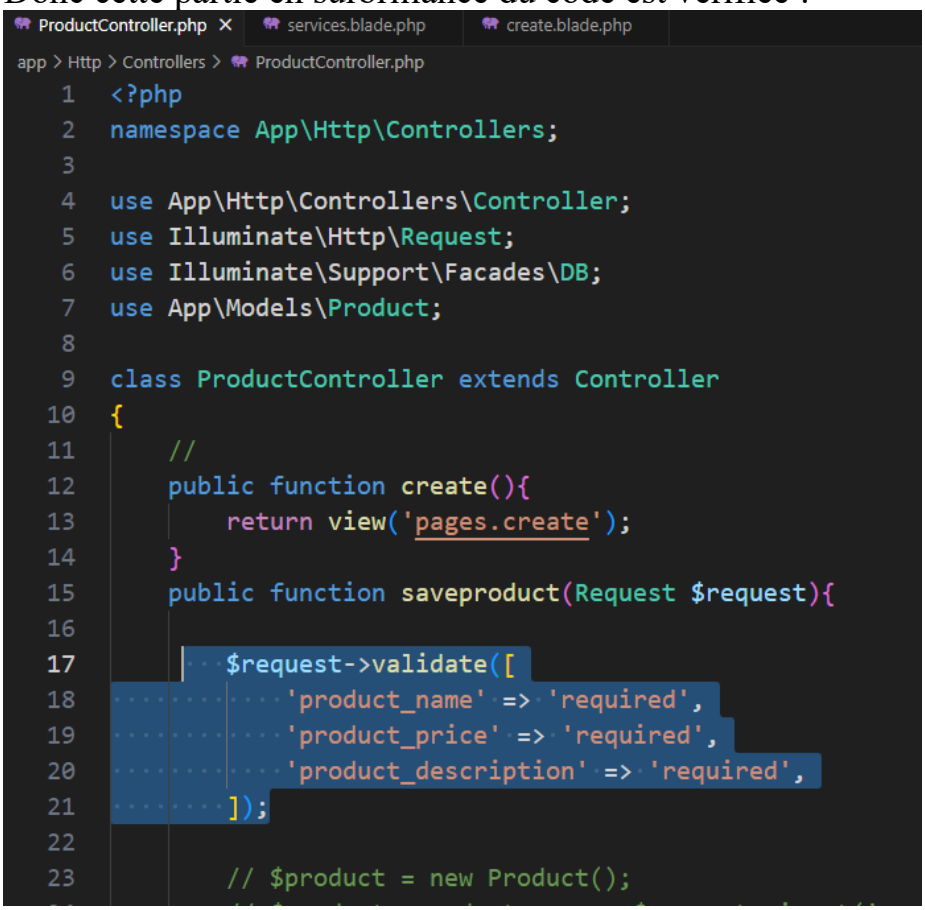


Deux messages sont affichés.
Il faudra remplir tous les champs pour que le formulaire soit valide.

-Si on remplit tous les champs et on clique sur Add Product



Le formulaire est envoyé mais retourne une page vide.
Donc cette partie en surbrillance du code est vérifiée :



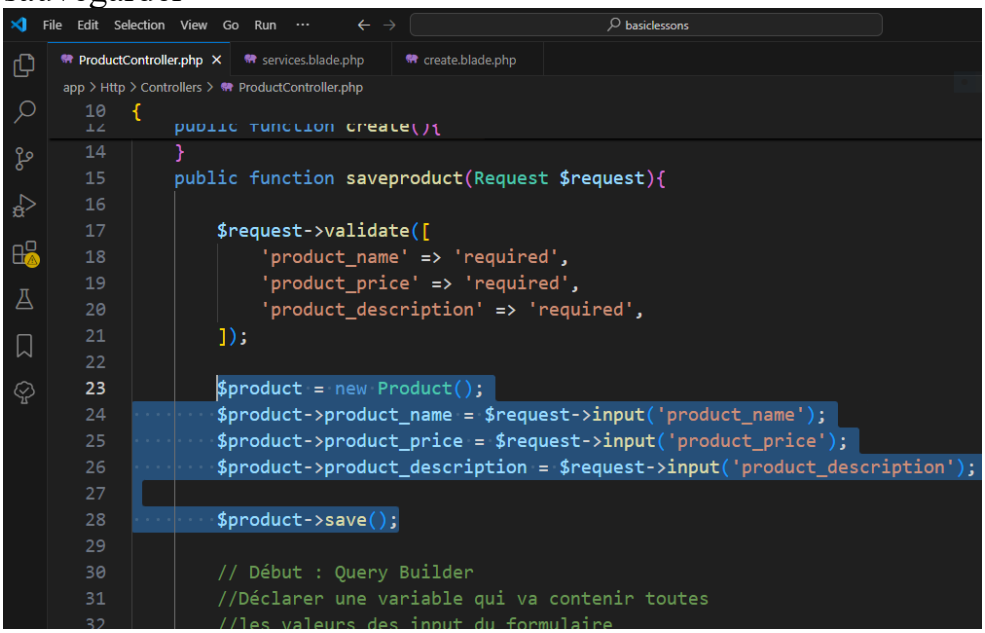
Il reste la partie **save** pour sauvegarder car si on vérifie au niveau de la base de données, aucune insertion n'est faite :

	id	product_name	product_price	product_description	created_at	updated_at
Éditer Copier Supprimer	1	Mangue	200	La mangue ddu jardin	2024-10-14 20:19:20	2024-10-14 20:24:18
Éditer Copier Supprimer	2	Pomme	100	La meilleur de la récolte	2024-10-14 20:30:02	2024-10-14 20:30:02
Éditer Copier Supprimer	3	Tomate	100	La meilleure tomate de la France.	2024-10-15 13:07:10	2024-10-15 13:07:10
Éditer Copier Supprimer	4	Banane	100	La banane de la Chine	2024-10-15 17:24:56	2024-10-15 17:24:56
Éditer Copier Supprimer	5	Poivron	150	Le meilleur poivron de la récolte.	2024-10-15 18:04:08	2024-10-15 18:04:08
Éditer Copier Supprimer	7	Carotte rouge	200	La carotte rouge du jardin de Toto.	NULL	NULL

Tout cocher Avec la sélection : Éditer Copier Supprimer Exporter

Pas de Raisins dans la base.

Maintenant dans le code, on va dé commenter la partie Eloquent pour sauvegarder



-Actualiser le navigateur et ensuite la base de données (phpMyAdmin) :

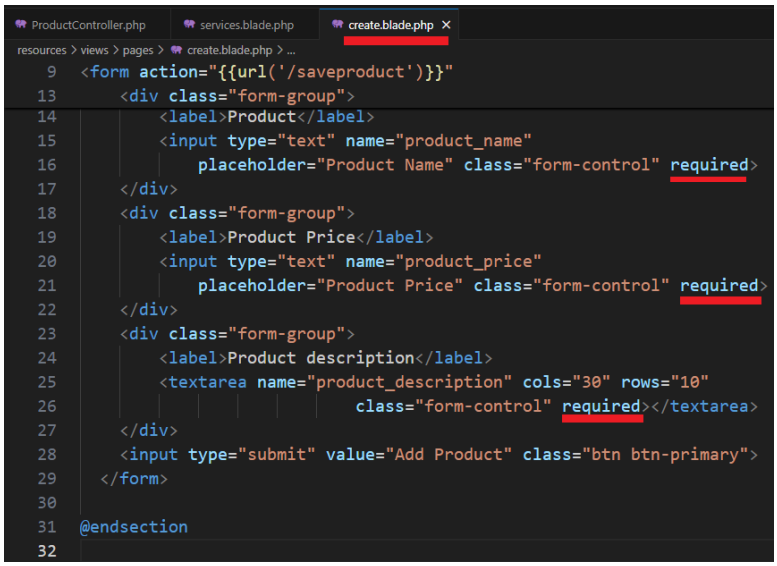
	id	product_name	product_price	product_description	created_at	updated_at
Éditer Copier Supprimer	1	Mangue	200	La mangue ddu jardin	2024-10-14 20:19:20	2024-10-14 20:24:18
Éditer Copier Supprimer	2	Pomme	100	La meilleur de la récolte	2024-10-14 20:30:02	2024-10-14 20:30:02
Éditer Copier Supprimer	3	Tomate	100	La meilleure tomate de la France.	2024-10-15 13:07:10	2024-10-15 13:07:10
Éditer Copier Supprimer	4	Banane	100	La banane de la Chine	2024-10-15 17:24:56	2024-10-15 17:24:56
Éditer Copier Supprimer	5	Poivron	150	Le meilleur poivron de la récolte.	2024-10-15 18:04:08	2024-10-15 18:04:08
Éditer Copier Supprimer	7	Carotte rouge	200	La carotte rouge du jardin de Toto.	NULL	NULL
Éditer Copier Supprimer	8	Raisin	150	Les raisins du jour.	2024-10-16 09:47:26	2024-10-16 09:47:26

Tout cocher Avec la sélection : Éditer Copier Supprimer Exporter

Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table | Trier par clé : Aucun(e)

L'insertion est maintenant dans la base de données (enregistrement 8).

NB : remettre le **required** dans **create.blade.php**


ProductController.php
services.blade.php
create.blade.php
resources > views > pages > create.blade.php > ...
<form action="{{url('/saveproduct')}}"

<label>Product</label>
<input type="text" name="product_name"
placeholder="Product Name" class="form-control" required>

<label>Product Price</label>
<input type="text" name="product_price"
placeholder="Product Price" class="form-control" required>

<label>Product description</label>
<textarea name="product_description" cols="30" rows="10"
class="form-control" required></textarea>

<input type="submit" value="Add Product" class="btn btn-primary">
</form>
@endsection